

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Hiển thị giao thông trong không gian 3D từ dữ liệu
mô phỏng SUMO**

ĐẶNG MINH HOÀNG

Hoang.DM225719@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Hiển thị giao thông trong không gian 3D từ dữ liệu
mô phỏng SUMO**

ĐẶNG MINH HOÀNG

Hoang.DM225719@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nhận được nhiều sự giúp đỡ và góp ý quý báu. Trước hết, em xin chân thành cảm ơn ThS. Nguyễn Tiến Thành đã tận tình hướng dẫn, định hướng nội dung và hỗ trợ em tháo gỡ các vướng mắc trong suốt quá trình nghiên cứu và triển khai. Em cũng xin cảm ơn các thầy cô của Trường Công nghệ Thông tin và Truyền thông — Đại học Bách khoa Hà Nội vì những kiến thức nền tảng và phương pháp học tập đã trang bị cho em. Em xin gửi lời cảm ơn tới gia đình và bạn bè vì luôn động viên, tạo điều kiện về thời gian và tinh thần để em hoàn thành đồ án đúng tiến độ. Những thiếu sót trong báo cáo là trách nhiệm của riêng em và em mong nhận được thêm góp ý để hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Nhu cầu quan sát và phân tích mô phỏng giao thông ngày càng tăng, tuy nhiên giao diện 2D truyền thống của SUMO còn hạn chế về khả năng trực quan hóa không gian, theo dõi đồng thời nhiều đối tượng và trình bày kịch bản cho người không chuyên. Một số giải pháp hiển thị 3D đã tồn tại nhưng thường phụ thuộc vào bản đồ tĩnh dựng sẵn, khó tái sử dụng cho nhiều kịch bản và ít hỗ trợ tương tác trực tiếp với phương tiện. Trong đồ án này, em lựa chọn hướng tiếp cận kết nối chuỗi công cụ SUMO — TraCI/Python — Unity: SUMO mô phỏng giao thông vi mô và sinh dữ liệu, Python dùng TraCI thu thập trạng thái theo từng bước thời gian và truyền sang Unity qua TCP/IP, còn Unity đảm nhận dựng cảnh và hiển thị 3D theo thời gian thực. Hướng tiếp cận này được chọn vì tận dụng được năng lực mô phỏng mạnh của SUMO đồng thời khai thác pipeline dựng hình, vật lý và tương tác của Unity. Trên cơ sở đó, hệ thống tự động dựng mạng đường từ dữ liệu mô phỏng, hiển thị phương tiện, đèn tín hiệu và người đi bộ, hỗ trợ hai chế độ vận hành là thời gian thực và phát lại, cùng các thao tác điều khiển camera, tạm dừng/tiếp tục và điều chỉnh tốc độ mô phỏng. Đóng góp nổi bật của đồ án là cơ chế cho phép người dùng chiếm quyền điều khiển một phương tiện và lái thủ công bằng vật lý, xử lý va chạm để sinh “xác xe” gây ùn ứ, và trả quyền điều khiển để xóa xe khỏi mô phỏng ngay lập tức; bên cạnh đó là cơ chế lưu trữ phiên mô phỏng hợp nhất phục vụ truy vết và phát lại. Hệ thống còn được tối ưu luồng truyền dữ liệu (gom truy vấn TraCI, nén dữ liệu trạng thái, giảm độ trễ gói tin) kèm cơ chế đo độ trễ đầu-cuối, và dựng thêm khối công trình từ bản đồ OpenStreetMap để tăng tính trực quan. Kết quả thực nghiệm trên các kịch bản sinh từ mê cung và từ bản đồ OpenStreetMap cho thấy hệ thống dựng cảnh đúng dữ liệu nguồn, tái hiện chuyển động ổn định và đáp ứng nhu cầu quan sát, phân tích trực quan.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 Công cụ quan sát đi kèm trình mô phỏng.....	5
2.1.2 Các ứng dụng trực quan hóa giao thông ba chiều.....	6
2.1.3 So sánh và rút ra yêu cầu	6
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã nhóm Chuẩn bị kịch bản và bản đồ.....	8
2.2.3 Biểu đồ use case phân rã nhóm Vận hành mô phỏng.....	9
2.2.4 Biểu đồ use case phân rã nhóm Quan sát và điều khiển.....	10
2.2.5 Biểu đồ use case phân rã nhóm Lái xe tương tác	11
2.3 Đặc tả chức năng	11
2.3.1 Đặc tả use case Nhập bản đồ từ OpenStreetMap.....	11
2.3.2 Đặc tả use case Sinh tuyến đường cho phương tiện	12
2.3.3 Đặc tả use case Chạy mô phỏng thời gian thực.....	12
2.3.4 Đặc tả use case Tạm dừng và tiếp tục mô phỏng.....	13
2.3.5 Đặc tả use case Chiếm quyền điều khiển và lái xe	13
2.3.6 Đặc tả use case Trả quyền điều khiển phương tiện	15
2.4 Yêu cầu phi chức năng	15

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	17
3.1 Tổng quan công nghệ và vai trò trong hệ thống.....	17
3.2 SUMO (Simulation of Urban Mobility)	18
3.2.1 Giới thiệu	18
3.2.2 Lý do lựa chọn SUMO.....	18
3.2.3 Các thành phần cấu hình và dữ liệu trong SUMO	18
3.2.4 Các lựa chọn thay thế và đánh giá.....	19
3.3 TraCI (Traffic Control Interface).....	19
3.3.1 Giới thiệu	19
3.3.2 Mô hình hoạt động client–server	19
3.3.3 Nhóm chức năng TraCI sử dụng trong đề án.....	20
3.3.4 Lựa chọn thay thế.....	21
3.4 Python cho điều phối và xử lý dữ liệu	21
3.4.1 Vai trò trong hệ thống.....	21
3.4.2 Chuẩn hóa theo định dạng trao đổi độc lập nền tảng	21
3.4.3 Tổ chức xử lý theo bước thời gian	21
3.4.4 Lựa chọn thay thế.....	21
3.5 Unity cho hiển thị 3D và tương tác	22
3.5.1 Giới thiệu và lý do lựa chọn	22
3.5.2 Pipeline hiển thị đối tượng giao thông	22
3.5.3 Hệ thống vật lý của Unity cho chế độ lái	22
3.5.4 Lựa chọn thay thế.....	22
3.6 Tích hợp SUMO–TraCI/Python–Unity.....	23
3.6.1 Phương thức trao đổi dữ liệu	23
3.6.2 Định dạng dữ liệu tối thiểu.....	23
3.6.3 Đồng bộ thời gian giữa bước mô phỏng và khung hình	23

3.6.4 Kết chương.....	24
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	25
4.1 Thiết kế kiến trúc.....	25
4.1.1 Lựa chọn kiến trúc phần mềm	25
4.1.2 Kiến trúc tổng quan.....	25
4.1.3 Thiết kế các mô-đun chính	26
4.2 Thiết kế chi tiết.....	28
4.2.1 Thiết kế module dựng bản đồ 3D	28
4.2.2 Máy trạng thái điều khiển phương tiện	29
4.2.3 Hai luồng dữ liệu giữa mô phỏng và hiển thị.....	30
4.2.4 Thiết kế module giao thông.....	31
4.2.5 Lưu trữ dữ liệu.....	33
4.2.6 Thiết kế giao diện người dùng	34
4.3 Xây dựng ứng dụng.....	36
4.3.1 Thư viện và công cụ sử dụng.....	36
4.3.2 Kết quả đạt được và thống kê mã nguồn	36
4.3.3 Minh họa các chức năng chính	36
4.4 Kiểm thử.....	36
4.5 Thực nghiệm và đánh giá.....	39
4.5.1 Môi trường và kịch bản thực nghiệm.....	39
4.5.2 Phương pháp đánh giá	39
4.5.3 Kết quả	39
4.5.4 Triển khai.....	40

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	41
5.1 Dựng bản đồ ba chiều chính xác từ SUMO và OpenStreetMap.....	41
5.1.1 Vấn đề.....	41
5.1.2 Giải pháp	41
5.1.3 Kết quả đạt được	43
5.2 Tạo phương tiện của người dùng và chiếm quyền điều khiển	44
5.2.1 Vấn đề.....	44
5.2.2 Giải pháp	44
5.2.3 Kết quả đạt được	44
5.3 Hệ thống va chạm giả lập tai nạn.....	44
5.3.1 Vấn đề.....	44
5.3.2 Giải pháp	45
5.3.3 Kết quả đạt được	45
5.4 Lưu trữ kịch bản và phát lại	45
5.4.1 Vấn đề.....	45
5.4.2 Giải pháp	45
5.4.3 Kết quả đạt được	46
5.5 Giao tiếp thời gian thực tin cậy bằng tách khung bản tin	46
5.5.1 Vấn đề.....	46
5.5.2 Giải pháp	46
5.5.3 Kết quả đạt được	46
5.6 Tối ưu hiệu năng hiển thị quy mô lớn.....	46
5.6.1 Vấn đề.....	46
5.6.2 Giải pháp	46
5.6.3 Kết quả đạt được	47
5.7 Kết chương.....	47

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	48
6.1 Kết luận.....	48
6.2 Hướng phát triển.....	49
TÀI LIỆU THAM KHẢO.....	51
PHỤ LỤC.....	53
A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH.....	53
A.1 Yêu cầu phần cứng và phần mềm.....	53
A.2 Cài đặt môi trường	53
A.3 Khởi chạy hệ thống	54
A.3.1 Mô phỏng thời gian thực	54
A.3.2 Phát lại kịch bản có sẵn (tiền kết xuất).....	55
A.4 Xử lý sự cố thường gặp	55
B. ĐẶC TẢ USE CASE.....	57
B.1 Đặc tả use case Tạo bản đồ từ lưới mê cung	57
B.2 Đặc tả use case Nạp kịch bản tự dựng từ netedit	58
B.3 Đặc tả use case Sinh tuyến đường cho người đi bộ.....	58
B.4 Đặc tả use case Cấu hình tham số mô phỏng.....	59
B.5 Đặc tả use case Chạy mô phỏng chế độ phát lại.....	59
B.6 Đặc tả use case Chạy kèm giao diện sumo-gui	60
B.7 Đặc tả use case Hiển thị phương tiện theo thời gian	60
B.8 Đặc tả use case Hiển thị đèn tín hiệu.....	61
B.9 Đặc tả use case Hiển thị người đi bộ	61
B.10 Đặc tả use case Điều khiển camera quan sát.....	62
B.11 Đặc tả use case Điều chỉnh tốc độ mô phỏng.....	62
B.12 Đặc tả use case Lọc đối tượng hiển thị theo vùng	63
B.13 Đặc tả use case Xử lý va chạm sinh xác xe.....	63

DANH MỤC HÌNH VẼ

Hình 2.1	Giao diện hai chiều nhìn từ trên cao của <code>sumo-gui</code>	5
Hình 2.2	Biểu đồ use case tổng quát	7
Hình 2.3	Use case nhóm Chuẩn bị kịch bản và bản đồ	9
Hình 2.4	Use case nhóm Vận hành mô phỏng	10
Hình 2.5	Use case nhóm Quan sát và điều khiển	10
Hình 2.6	Use case nhóm Lái xe tương tác	11
Hình 2.7	Biểu đồ hoạt động use case Chiếm quyền và lái xe	14
Hình 2.8	Biểu đồ hoạt động use case Trả quyền điều khiển	15
Hình 3.1	Kiến trúc tổng quát hệ thống mô phỏng và hiển thị 3D	17
Hình 3.2	Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI	20
Hình 4.1	Kiến trúc chi tiết với ba kênh TCP và hai luồng dữ liệu	26
Hình 4.2	Sơ đồ gói các nhóm mô-đun hai phía và quan hệ phụ thuộc	27
Hình 4.3	Sơ đồ lớp module dựng bản đồ 3D phía Unity	29
Hình 4.4	Máy trạng thái điều khiển phương tiện	30
Hình 4.5	Trình tự trao đổi thông điệp giữa mô phỏng và hiển thị trong một bước	31
Hình 4.6	Sơ đồ lớp các lớp chủ đạo phía Unity (nhóm quản lý phương tiện và điều khiển lái).	32
Hình 4.7	Sơ đồ lớp và mô-đun giao thông phía Python server	33
Hình 4.8	Cấu trúc một thư mục phiên tiêu biểu, đặt tên theo bản đồ và thời điểm chạy. Tập <code>trips.csv</code> đổi thành <code>trips-ped.csv</code> khi kịch bản có người đi bộ.	34
Hình 4.9	Bố cục giao diện người dùng trong cảnh ba chiều	35
Hình 4.10	Minh họa cảnh giao thông ba chiều trong Unity: góc nhìn từ trên cao	37
Hình 4.11	Minh họa chế độ lái xe thủ công: từ quan sát đến chiếm quyền điều khiển	37
Hình 4.12	Minh họa xử lý va chạm: xác xe nằm lại gây ùn ứ dòng giao thông	38
Hình 5.1	Dựng dải mặt đường tám đường gấp khúc và hiệu chỉnh tại khúc cua (bản nháp)	42
Hình 5.2	Tam giác hóa đa giác nút giao rồi đùn thành khối ba chiều (bản nháp)	43

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các công cụ quan sát mô phỏng giao thông	6
Bảng 2.2	Danh sách use case của hệ thống	8
Bảng 2.3	Đặc tả use case UC2 – Nhập bản đồ từ OpenStreetMap	12
Bảng 2.4	Đặc tả use case UC4 – Sinh tuyến đường cho phương tiện . . .	12
Bảng 2.5	Đặc tả use case UC7 – Chạy mô phỏng thời gian thực	13
Bảng 2.6	Đặc tả use case UC11 – Tạm dừng và tiếp tục mô phỏng . . .	13
Bảng 2.7	Đặc tả use case UC14 – Chiếm quyền điều khiển và lái xe . .	14
Bảng 2.8	Đặc tả use case UC15 – Trả quyền điều khiển phương tiện . .	15
Bảng 3.1	Đầu vào và đầu ra của SUMO trong hệ thống	19
Bảng 3.2	Một số trường dữ liệu trao đổi giữa Python và Unity	23
Bảng 4.1	Các nhóm mô-đun chính phía Unity	26
Bảng 4.2	Danh sách thư viện và công cụ sử dụng	36
Bảng 4.3	Thống kê quy mô mã nguồn	36
Bảng 4.4	Một số trường hợp kiểm thử tiêu biểu	38
Bảng 4.5	Kết quả hiệu năng theo quy mô kịch bản (máy bàn)	39
Bảng 4.6	Kết quả hiệu năng kịch bản OSM trên máy tính xách tay . . .	39
Bảng A.1	Cấu hình hai máy tính dùng để phát triển và kiểm thử	53

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
BFS	Thuật toán duyệt đồ thị theo chiều rộng, dùng để lọc cặp OD khả thi (Breadth-First Search)
CSV	Định dạng tệp ghi log hành trình phương tiện (Comma-Separated Values)
FPS	Số khung hình mỗi giây của tăng hiển thị (Frames Per Second)
GUI	Giao diện đồ họa người dùng (Graphical User Interface)
JSON	Định dạng trao đổi dữ liệu giữa server và Unity (JavaScript Object Notation)
OD	Cặp điểm đầu–cuối của một chuyến đi (Origin–Destination)
OSM	Dữ liệu bản đồ địa lý mã nguồn mở (OpenStreetMap)
SUMO	Trình mô phỏng giao thông đô thị mã nguồn mở (Simulation of Urban Mobility)
TAZ	Vùng gán giao thông; sử dụng qua tùy chọn <code>-junction-taz</code> của SUMO (Traffic Assignment Zone)
TCP/IP	Giao thức truyền dữ liệu theo luồng giữa Python và Unity (Transmission Control Protocol / Internet Protocol)
TraCI	Giao diện điều khiển và truy vấn mô phỏng giao thông theo bước thời gian (Traffic Control Interface)

Thuật ngữ	Ý nghĩa
VRP	Bài toán định tuyến phương tiện (Vehicle Routing Problem)
XML	Định dạng tệp cấu hình và mạng đường của SUMO (eXtensible Markup Language)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh đô thị hóa nhanh và mật độ phương tiện gia tăng, bài toán tổ chức và quản lý giao thông ngày càng trở nên phức tạp, đòi hỏi các phương pháp phân tích có căn cứ dữ liệu để hỗ trợ quy hoạch, đánh giá phương án tổ chức giao thông và kiểm chứng các giả thuyết kỹ thuật. Trên thực tế, việc thử nghiệm trực tiếp ngoài hiện trường thường tốn kém, khó kiểm soát biến số, và có thể tiềm ẩn rủi ro về nhân mạng. Vì vậy, mô phỏng giao thông trở thành một công cụ quan trọng, giúp tái hiện nhiều kịch bản khác nhau trong điều kiện có thể kiểm soát, từ đó hỗ trợ quá trình ra quyết định.

Trong nhóm các công cụ mô phỏng, SUMO (Simulation of Urban MObility) [1], [2] là một nền tảng mô phỏng vi mô được sử dụng rộng rãi, cho phép mô tả mạng đường, luồng phương tiện và cơ chế điều khiển (như tín hiệu đèn) với mức độ chi tiết cao. Kết quả mô phỏng có thể cung cấp dữ liệu theo thời gian về vị trí, tốc độ, hướng di chuyển của phương tiện, cũng như các thông tin liên quan đến trạng thái mạng. Tuy nhiên, dữ liệu đầu ra này thường có khối lượng lớn và giàu tính kỹ thuật, khiến việc quan sát và khai thác hiệu quả trở thành một thách thức, đặc biệt khi người dùng cần đánh giá trực quan cấu trúc không gian hoặc diễn biến ở các khu vực có hình học phức tạp.

Việc quan sát kết quả mô phỏng bằng giao diện 2D truyền thống thường đáp ứng tốt các thao tác cơ bản, nhưng vẫn gặp hạn chế khi cần: (i) nhận thức không gian tại các nút giao nhiều nhánh hoặc có tổ chức làn phức tạp; (ii) theo dõi đồng thời nhiều đối tượng trong bối cảnh mật độ cao; và (iii) trình bày kết quả cho nhóm người dùng không chuyên hoặc trong các hoạt động trao đổi, thuyết minh phương án. Bản chất của góc nhìn 2D là góc nhìn từ trên cao: nó cho phép người quản lý giám sát giao thông thấy được toàn cảnh mạng đường và dòng phương tiện, nhưng lại không tái hiện được góc nhìn của người trực tiếp tham gia giao thông — tức là cách mà đường sá, biển báo, tín hiệu và các phương tiện khác hiện ra trước mắt một người đang di chuyển trong mạng. Trong các trường hợp này, góc nhìn 3D có thể giúp người quan sát nắm bắt tốt hơn mối quan hệ không gian giữa đường, làn, phương tiện và tín hiệu điều khiển, đồng thời tạo điều kiện cho các hình thức tương tác và trải nghiệm theo góc nhìn người tham gia giao thông.

Mặc dù vậy, các giải pháp hiển thị 3D trong thực tế thường gặp những rào cản như phụ thuộc vào bản đồ dựng sẵn (tĩnh), cần nhiều thao tác thủ công khi thay đổi kịch bản, hoặc khó mở rộng khi số lượng đối tượng và loại đối tượng tăng lên.

Ngoài ra, phần lớn giải pháp chỉ dừng ở mức quan sát thụ động, chưa cho phép người dùng tương tác trực tiếp với phương tiện trong cảnh để khảo sát các tình huống giả định. Việc chuyển dữ liệu mô phỏng sang môi trường 3D cũng phát sinh các vấn đề kỹ thuật như ánh xạ hình học từ mô tả mạng đường (đa tuyến) sang mô hình cảnh, chuyển đổi hệ tọa độ, đồng bộ thời gian giữa bước mô phỏng và khung hình hiển thị, và đảm bảo hiệu năng khi cập nhật số lượng lớn đối tượng theo thời gian.

Do đó, việc xây dựng một hệ thống hiển thị giao thông 3D dựa trên dữ liệu đầu ra của SUMO, có khả năng tự động dựng cảnh theo bản đồ mô phỏng, cập nhật trạng thái theo thời gian và cho phép tương tác, là cần thiết để nâng cao hiệu quả quan sát, phân tích và truyền đạt kết quả mô phỏng. Hệ thống này đóng vai trò cầu nối giữa dữ liệu mô phỏng (mang tính kỹ thuật) và nhu cầu khai thác trực quan, giúp người dùng tiếp cận, kiểm chứng và trình bày kịch bản giao thông một cách trực quan và nhất quán.

1.2 Mục tiêu và phạm vi đề tài

Từ các vấn đề nêu ở phần 1.1, đề án hướng tới mục tiêu xây dựng một hệ thống hiển thị mô phỏng giao thông trong không gian 3D, trong đó dữ liệu được lấy trực tiếp từ quá trình chạy mô phỏng của SUMO. Trọng tâm của hệ thống là hình thành một quy trình chuyển đổi dữ liệu mạng đường và dữ liệu mô phỏng sang các đối tượng 3D trong Unity một cách nhất quán, đồng thời bổ sung khả năng tương tác để người dùng không chỉ quan sát mà còn có thể can thiệp vào diễn biến giao thông.

Về mục tiêu cụ thể, đề án tập trung vào bốn nhóm mục tiêu chính. Thứ nhất, xây dựng cơ chế dựng mạng đường trong không gian 3D từ dữ liệu của SUMO, bảo đảm các đối tượng được dựng bám sát dữ liệu nguồn; mạng đường có thể được sinh tự động từ lưới mê cung phục vụ kiểm thử hoặc nhập từ bản đồ thực tế OpenStreetMap [3]. Thứ hai, xây dựng cơ chế thu thập dữ liệu động theo thời gian thông qua TraCI [4], nhằm lấy trạng thái của phương tiện, đèn tín hiệu và người đi bộ theo từng bước mô phỏng. Thứ ba, thiết kế cơ chế cập nhật và hiển thị trạng thái trong Unity theo thời gian, bảo đảm chuyển động thể hiện đúng quỹ đạo tổng quát và duy trì hiệu năng ổn định khi số lượng đối tượng tăng. Thứ tư, xây dựng khả năng tương tác cho phép người dùng chiếm quyền điều khiển một phương tiện và lái thủ công bằng vật lý, xử lý va chạm và trả quyền điều khiển để phương tiện hòa trở lại dòng giao thông do SUMO quản lý.

Trong phạm vi của đề tài, hệ thống tập trung vào các thành phần cốt lõi gồm: (i) dựng cảnh 3D từ dữ liệu mạng đường, bao gồm các đoạn đường, các yếu tố hình học phục vụ quan sát, và khối công trình (tòa nhà) khi nhập từ bản đồ OpenStreetMap;

(ii) biểu diễn phương tiện và người đi bộ như các thực thể di chuyển theo trạng thái thời gian; (iii) biểu diễn các tín hiệu điều khiển ở mức phù hợp với dữ liệu có thể truy xuất; (iv) chuẩn hóa, ánh xạ dữ liệu sang hệ tọa độ của Unity để bảo đảm tính nhất quán; và (v) hỗ trợ tương tác lái xe ở mức một người dùng cho mỗi phiên mô phỏng. Phần đánh giá được thực hiện thông qua các kịch bản mô phỏng mẫu nhằm kiểm chứng tính đúng đắn của việc ánh xạ dữ liệu, khả năng tái hiện diễn biến theo thời gian và tính ổn định của hiển thị.

Ngoài phạm vi của đề án là các bài toán tối ưu điều khiển giao thông, hiệu chỉnh hay tham số hóa mô hình mô phỏng của SUMO, cũng như hỗ trợ nhiều người dùng đồng thời trong cùng một phiên. Trong phạm vi này, hệ thống đóng vai trò là lớp trực quan hóa, tương tác và hỗ trợ phân tích dựa trên dữ liệu mô phỏng sẵn có, qua đó tăng khả năng quan sát và trình bày kịch bản thay vì can thiệp vào thuật toán mô phỏng.

1.3 Định hướng giải pháp

Để hiện thực hóa mục tiêu ở phần 1.2, đề án lựa chọn hướng tiếp cận dựa trên chuỗi công cụ: SUMO dùng để mô phỏng giao thông vi mô và sinh dữ liệu; Python kết hợp TraCI dùng để truy xuất trạng thái mô phỏng theo thời gian; Unity dùng để dựng cảnh, hiển thị 3D và xử lý tương tác. Lựa chọn này phù hợp vì SUMO là phần mềm mã nguồn mở, được sử dụng rộng rãi trong giới nghiên cứu và học thuật, đồng thời cung cấp dữ liệu mô phỏng có cấu trúc rõ ràng và phổ biến; trong khi Unity là môi trường phát triển 3D mạnh, hỗ trợ dựng cảnh, quản lý đối tượng, camera, vật lý và tương tác, qua đó nâng cao khả năng quan sát và can thiệp trực quan. Sự kết hợp giữa hai nền tảng này hướng tới phục vụ đồng thời hai nhóm người dùng: nhóm nghiên cứu và học tập vốn quen thuộc với SUMO nay có thêm khả năng quan sát, trình bày kịch bản một cách trực quan; và nhóm phát triển trò chơi (game) vốn quen thuộc với Unity nay có thể tận dụng dữ liệu giao thông được mô phỏng sát thực tế làm nền tảng cho các sản phẩm tương tác.

Trên cơ sở đó, giải pháp được triển khai theo một quy trình tổng quát gồm các thành phần chính. Thứ nhất, mô-đun chuẩn bị dữ liệu sinh mạng đường (từ mô hình hoặc từ OpenStreetMap) cùng tuyến đường cho phương tiện và người đi bộ, tạo thành kịch bản mô phỏng đầu vào cho SUMO. Thứ hai, mô-đun thu thập dữ liệu động kết nối với phiên mô phỏng qua TraCI để lấy trạng thái theo từng bước, sau đó chuẩn hóa thành một định dạng trao đổi thống nhất và truyền sang Unity qua TCP/IP. Thứ ba, mô-đun hiển thị trong Unity nhận dữ liệu, ánh xạ hệ tọa độ, tạo và cập nhật các đối tượng 3D, đồng thời nội suy chuyển động để hiển thị mượt. Thứ tư, mô-đun tương tác cho phép người dùng chiếm quyền điều khiển một phương

tiện, lái bằng vật lý, xử lý va chạm để sinh trạng thái “xác xe”, và trả quyền điều khiển để xóa xe khỏi mô phỏng ngay lập tức.

Trong quá trình triển khai, một yêu cầu quan trọng là bảo đảm tính nhất quán của dữ liệu giữa các mô-đun: dữ liệu hình học mạng đường cần dùng chung chuẩn tọa độ với dữ liệu phương tiện; các đối tượng trong Unity cần có vòng đời phù hợp với trạng thái mô phỏng (tạo mới khi xuất hiện, cập nhật khi di chuyển, loại bỏ khi rời mạng). Đồng thời, để bảo đảm khả năng mở rộng, giải pháp ưu tiên tách biệt phần thu thập/chuẩn hóa dữ liệu (Python) khỏi phần hiển thị (Unity), giúp giảm phụ thuộc và thuận tiện khi thay đổi kịch bản hoặc nền tảng hiển thị.

Đóng góp chính của đề án là đề xuất và hiện thực một quy trình chuyển đổi dữ liệu mô phỏng từ SUMO sang môi trường hiển thị 3D trong Unity theo hướng tự động, giảm thao tác thủ công khi thay đổi kịch bản; đồng thời bổ sung khả năng tương tác lái xe, xử lý va chạm và trả quyền điều khiển vốn ít gặp ở các giải pháp hiển thị thông thường. Kết quả thử nghiệm trên các kịch bản mẫu cho thấy hệ thống có thể dựng được mạng đường từ dữ liệu đầu vào, biểu diễn chuyển động của phương tiện và người đi bộ một cách ổn định, và cho phép người dùng can thiệp vào diễn biến giao thông, tạo cơ sở cho việc quan sát và phân tích trực quan.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau. Chương 2 trình bày khảo sát hiện trạng và phân tích yêu cầu của hệ thống, trong đó so sánh các công cụ hiển thị mô phỏng hiện có, xác định các tác nhân, mô tả tổng quan chức năng qua biểu đồ use case và đặc tả chi tiết các use case quan trọng cùng các yêu cầu phi chức năng. Chương 3 giới thiệu nền tảng lý thuyết và các công nghệ được sử dụng, bao gồm SUMO, TraCI, Python và Unity, đồng thời phân tích vai trò của từng thành phần và lý do lựa chọn so với các phương án thay thế. Chương 4 trình bày phân tích thiết kế, triển khai và đánh giá hệ thống, tập trung vào kiến trúc tổng thể, các mô-đun chính, cơ chế đồng bộ dữ liệu và kết quả thực nghiệm trên các kịch bản mẫu. Chương 5 trình bày các giải pháp kỹ thuật và đóng góp nổi bật của đề án, trong đó có cơ chế giao tiếp thời gian thực, cơ chế chiếm quyền điều khiển và xử lý va chạm, cũng như cơ chế trả quyền và lưu trữ phiên mô phỏng. Cuối cùng, Chương 6 tổng kết các kết quả đạt được, các đóng góp chính của đề án và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

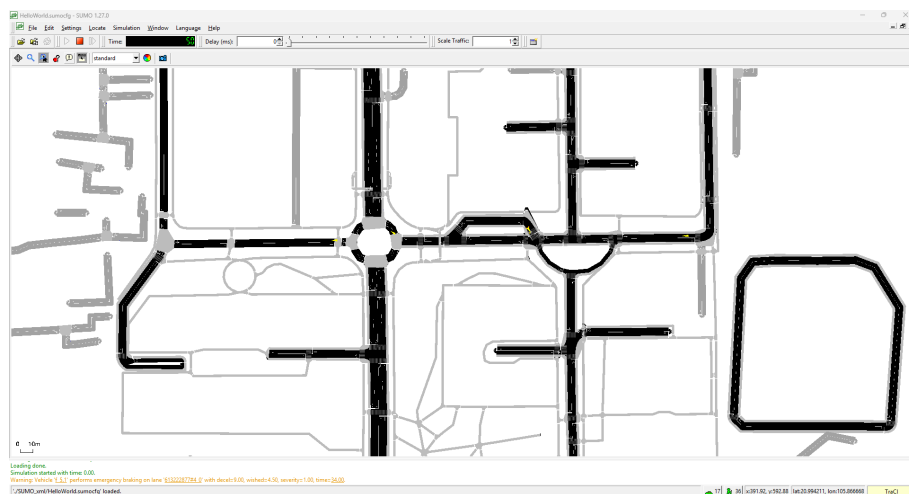
Chương 1 đã giới thiệu bối cảnh, mục tiêu và định hướng giải pháp của đề tài. Chương này tiến hành khảo sát hiện trạng các công cụ quan sát mô phỏng giao thông hiện có, phân tích yêu cầu của hệ thống và mô tả các chức năng dưới dạng biểu đồ use case. Nội dung gồm bốn phần: khảo sát hiện trạng (mục 2.1), tổng quan chức năng qua biểu đồ use case (mục 2.2), đặc tả chi tiết các use case quan trọng (mục 2.3) và các yêu cầu phi chức năng (mục 2.4).

2.1 Khảo sát hiện trạng

Việc khảo sát hiện trạng được thực hiện dựa trên ba nguồn: nhu cầu của người dùng khi quan sát mô phỏng giao thông, các công cụ quan sát đi kèm trình mô phỏng, và các ứng dụng trực quan hóa giao thông ba chiều tương tự. Mục tiêu là xác định các hạn chế còn tồn tại, từ đó rút ra những tính năng cần phát triển.

2.1.1 Công cụ quan sát đi kèm trình mô phỏng

SUMO cung cấp sẵn công cụ quan sát `sumo-gui` với giao diện hai chiều theo góc nhìn từ trên cao. Công cụ này hiển thị đầy đủ mạng đường, phương tiện, làn và trạng thái đèn tín hiệu, đồng thời cho phép phóng to, thu nhỏ và truy vấn thuộc tính của từng đối tượng. Ưu điểm của `sumo-gui` là bám sát dữ liệu mô phỏng, nhẹ và ổn định. Hình 2.1 minh họa giao diện hai chiều của công cụ này. Tuy nhiên, do bản chất là góc nhìn từ trên cao, công cụ này phù hợp cho việc giám sát tổng thể nhưng khó tái hiện quan hệ không gian theo chiều cao và không cung cấp góc nhìn của người trực tiếp tham gia giao thông.



Hình 2.1: Giao diện hai chiều nhìn từ trên cao của `sumo-gui`

2.1.2 Các ứng dụng trực quan hóa giao thông ba chiều

Trong nhóm các giải pháp hiển thị ba chiều, có thể kể đến hai hướng tiêu biểu. Hướng thứ nhất là các nền tảng mô phỏng phục vụ xe tự hành như CARLA, cung cấp môi trường ba chiều chân thực cùng mô hình cảm biến. Tuy nhiên, các nền tảng này yêu cầu tài nguyên lớn, tập trung vào mô phỏng cảm biến cho một số ít phương tiện thay vì hiển thị dòng giao thông quy mô lớn từ một trình mô phỏng vi mô. Hướng thứ hai là các công cụ chuyển dữ liệu mô phỏng sang môi trường ba chiều để phát lại; các công cụ này giúp quan sát trực quan hơn nhưng thường phụ thuộc vào bản đồ dựng sẵn, ít hỗ trợ thay đổi kịch bản nhanh và hiếm khi cho phép người dùng can thiệp, điều khiển trực tiếp một phương tiện trong cảnh.

2.1.3 So sánh và rút ra yêu cầu

Bảng 2.1 so sánh các công cụ hiện có với giải pháp đề xuất của đề án theo các tiêu chí quan trọng đối với nhu cầu quan sát và phân tích.

Bảng 2.1: So sánh các công cụ quan sát mô phỏng giao thông

Tiêu chí	sumo-gui	CARLA	Đề án
Hiển thị ba chiều	Không	Có	Có
Dữ liệu từ mô phỏng vi mô	Có	Hạn chế	Có
Dựng cảnh tự động theo kịch bản	Có	Hạn chế	Có
Hiển thị quy mô lớn nhiều đối tượng	Có	Hạn chế	Có
Góc nhìn người tham gia giao thông	Không	Có	Có
Tương tác lái xe trong cảnh	Không	Có	Có
Yêu cầu tài nguyên	Thấp	Cao	Trung bình

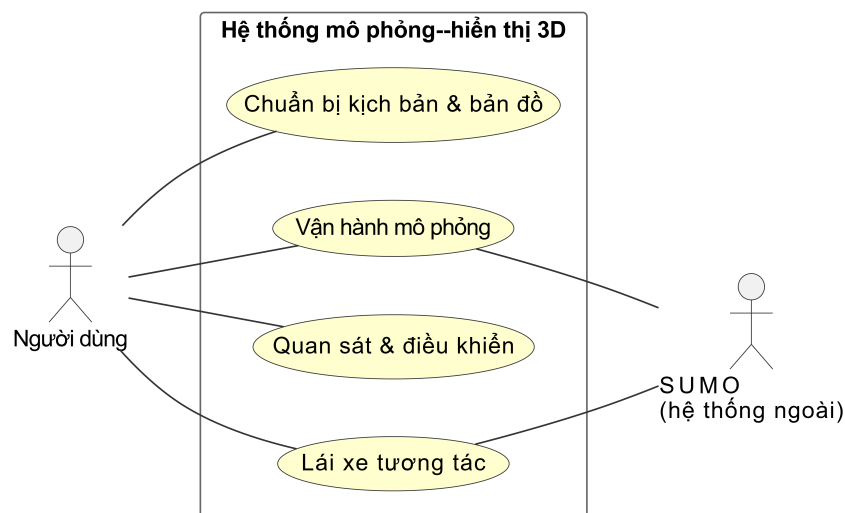
Từ bảng so sánh, có thể thấy khoảng trống chính nằm ở khả năng kết hợp đồng thời ba yếu tố: hiển thị ba chiều một lượng lớn phương tiện mà vẫn giữ được mức chi tiết tới từng xe của mô phỏng vi mô, dựng cảnh tự động theo kịch bản, và cho phép tương tác theo góc nhìn người tham gia giao thông với yêu cầu tài nguyên vừa phải. Trên cơ sở đó, hệ thống cần phát triển các tính năng quan trọng gồm: dựng cảnh ba chiều tự động từ dữ liệu mạng đường; hiển thị phương tiện, đèn tín hiệu và người đi bộ theo thời gian; cung cấp các thao tác quan sát như điều khiển camera, tạm dừng và điều chỉnh tốc độ; và cho phép người dùng chiếm quyền điều khiển, lái thử một phương tiện trong cảnh.

2.2 Tổng quan chức năng

2.2.1 Biểu đồ use case tổng quát

Hệ thống có một tác nhân chính là **Người dùng**. Đây là người chuẩn bị kịch bản, khởi chạy và quan sát mô phỏng, thực hiện các thao tác điều khiển quá trình chạy và xuất kết quả; ngoài ra, cùng người dùng này có thể chiếm quyền điều khiển một phương tiện để lái thử công. Hệ thống vận hành theo mô hình một người dùng cho mỗi phiên mô phỏng, do đó vai trò “lái xe” không phải một người riêng biệt mà chỉ là một chế độ tương tác của chính người dùng đó. Bên cạnh đó, **trình mô phỏng SUMO** đóng vai trò một tác nhân phụ (hệ thống ngoài) chịu trách nhiệm sinh dữ liệu mô phỏng vi mô và thực thi các lệnh điều khiển nhận được.

Các use case của hệ thống được nhóm thành bốn nhóm chức năng: chuẩn bị kịch bản và bản đồ, vận hành mô phỏng, quan sát và điều khiển, và lái xe tương tác. Biểu đồ use case tổng quát ở Hình 2.2 thể hiện quan hệ giữa tác nhân với bốn nhóm chức năng này.



Hình 2.2: Biểu đồ use case tổng quát

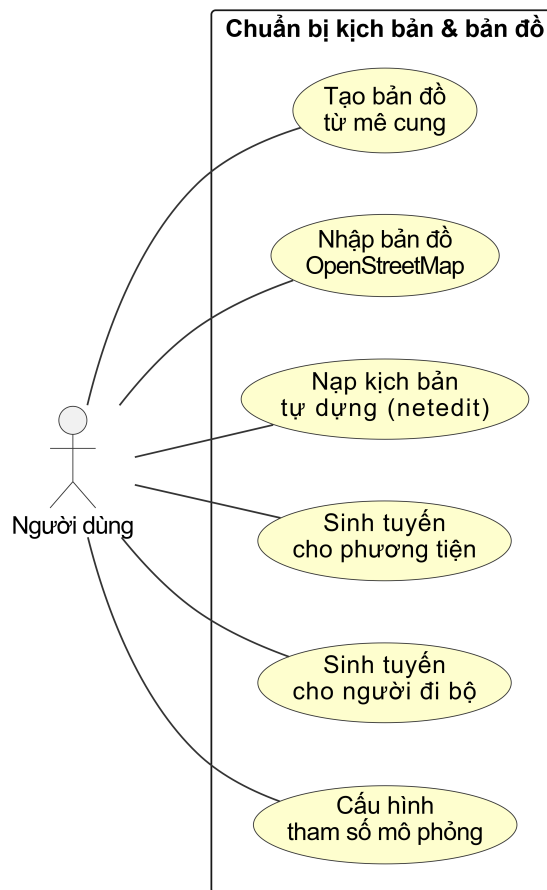
Bảng 2.2 liệt kê toàn bộ các use case của hệ thống theo bốn nhóm, làm cơ sở cho các biểu đồ phân rã ở các mục tiếp theo.

Bảng 2.2: Danh sách use case của hệ thống

Mã	Use case	Tác nhân
<i>A. Chuẩn bị kịch bản & bản đồ</i>		
UC1	Tạo bản đồ từ lưới mê cung	Người dùng
UC2	Nhập bản đồ từ OpenStreetMap	Người dùng
UC3	Nạp kịch bản tự động từ netedit	Người dùng
UC4	Sinh tuyến đường cho phương tiện	Người dùng
UC5	Sinh tuyến đường cho người đi bộ	Người dùng
UC6	Cấu hình tham số mô phỏng	Người dùng
<i>B. Vận hành mô phỏng</i>		
UC7	Chạy mô phỏng thời gian thực	Người dùng, SUMO
UC8	Chạy mô phỏng chế độ phát lại	Người dùng, SUMO
UC9	Chạy kèm giao diện sumo-gui	Người dùng, SUMO
<i>C. Quan sát & điều khiển</i>		
UC10	Điều khiển camera quan sát	Người dùng
UC11	Tạm dừng / tiếp tục mô phỏng	Người dùng
UC12	Điều chỉnh tốc độ mô phỏng	Người dùng
UC13	Lọc đối tượng hiển thị theo vùng	Người dùng
<i>D. Lái xe tương tác</i>		
UC14	Chiếm quyền điều khiển và lái xe	Người dùng, SUMO
UC15	Trả quyền điều khiển phương tiện	Người dùng, SUMO
UC16	Xử lý va chạm sinh xác xe	Người dùng, SUMO

2.2.2 Biểu đồ use case phân rõ nhóm Chuẩn bị kịch bản và bản đồ

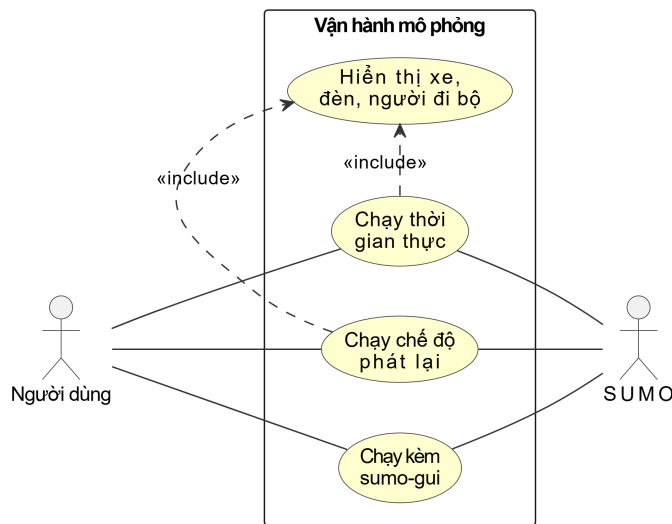
Nhóm này gồm các use case phục vụ tạo dữ liệu đầu vào cho mô phỏng. Người vận hành tạo mạng đường (từ lưới mê cung hoặc từ bản đồ OpenStreetMap), sinh tuyến đường cho phương tiện và người đi bộ, và cấu hình các tham số mô phỏng. Hình 2.3 thể hiện các use case của nhóm này.



Hình 2.3: Use case nhóm Chuẩn bị kịch bản và bản đồ

2.2.3 Biểu đồ use case phân rã nhóm Vận hành mô phỏng

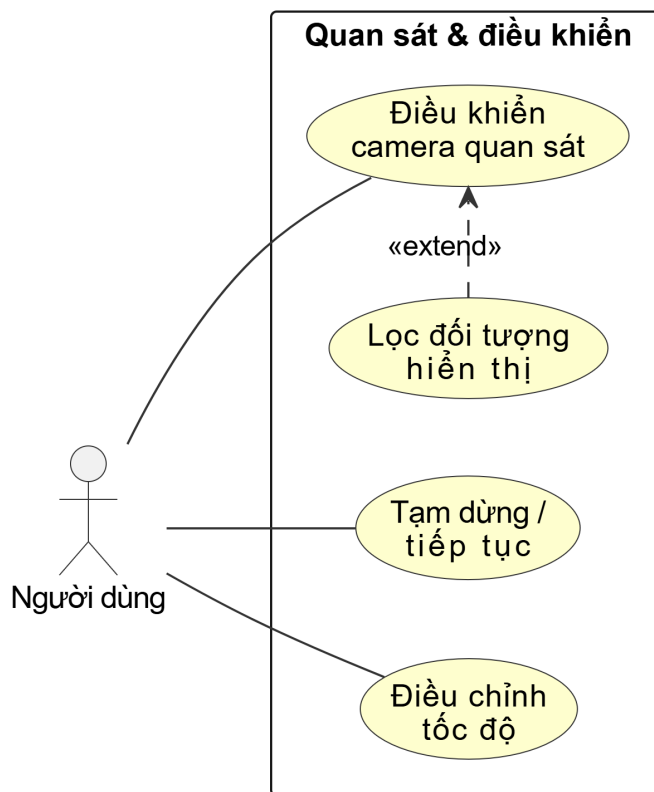
Nhóm này gồm các use case khởi chạy mô phỏng theo các chế độ khác nhau: thời gian thực, phát lại, và kèm giao diện `sumo-gui` để đối chiếu. Các use case này tương tác với trình mô phỏng SUMO. UC7 và UC8 tự động bao gồm việc hiển thị phương tiện, đèn tín hiệu và người đi bộ dưới dạng quan hệ «*include*». Hình 2.4 thể hiện nhóm chức năng này.



Hình 2.4: Use case nhóm Vận hành mô phỏng

2.2.4 Biểu đồ use case phân rã nhóm Quan sát và điều khiển

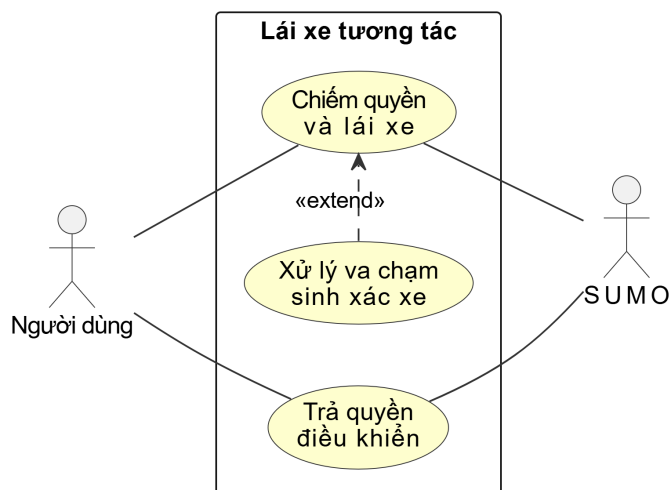
Nhóm này gồm các use case quan sát và kiểm soát quá trình chạy mô phỏng: điều khiển camera, tạm dừng/tiếp tục và điều chỉnh tốc độ. UC13 (lọc đối tượng hiển thị theo vùng quan sát) được mô hình hóa là quan hệ «*extend*» của UC10: chức năng lọc có thể bật hoặc tắt, nên chỉ mở rộng UC10 khi đang được kích hoạt. Hình 2.5 thể hiện nhóm chức năng này.



Hình 2.5: Use case nhóm Quan sát và điều khiển

2.2.5 Biểu đồ use case phân rã nhóm Lái xe tương tác

Nhóm này gồm các use case tương tác trực tiếp với phương tiện trong cảnh mô phỏng: chiếm quyền điều khiển một phương tiện để lái thử công và trả quyền về cho SUMO. UC16 (xử lý va chạm sinh xác xe) được mô hình hóa là quan hệ «*extend*» của UC14: khi đang lái xe, nếu xảy ra va chạm thì hệ thống tự động kích hoạt luồng xử lý xác xe. Hình 2.6 thể hiện nhóm này.



Hình 2.6: Use case nhóm Lái xe tương tác

2.3 Đặc tả chức năng

Phần này đặc tả chi tiết sáu use case quan trọng nhất, được lựa chọn vì nằm trên luồng nghiệp vụ chính và có nhiều luồng sự kiện phát sinh. Các use case còn lại được đặc tả đầy đủ trong Phụ lục B.

2.3.1 Đặc tả use case Nhập bản đồ từ OpenStreetMap

Bảng dưới đây đặc tả use case UC2 – khởi đầu luồng chuẩn bị kịch bản khi người dùng có dữ liệu bản đồ thực tế.

Bảng 2.3: Đặc tả use case UC2 – Nhập bản đồ từ OpenStreetMap

Tên	Nhập bản đồ từ OpenStreetMap
Tác nhân	Người dùng
Mô tả	Hệ thống chuyển một tệp bản đồ OpenStreetMap thành mạng đường và kịch bản mô phỏng.
Tiền điều kiện	Người dùng có tệp bản đồ OpenStreetMap hợp lệ.
Luồng chính	(1) Người dùng chọn tệp bản đồ. (2) Hệ thống chuyển đổi thành mạng đường. (3) Hệ thống sinh tuyến cho phương tiện và người đi bộ. (4) Hệ thống tạo tệp cấu hình mô phỏng.
Luồng phụ	(2a) Bản đồ lỗi hoặc không dựng được mạng: hệ thống báo lỗi và dừng quá trình.
Hậu điều kiện	Kịch bản mô phỏng từ bản đồ thực tế được tạo, sẵn sàng chạy.

2.3.2 Đặc tả use case Sinh tuyến đường cho phương tiện

Bảng dưới đây đặc tả use case UC4 – bước tiếp theo sau khi đã có mạng đường, sinh các tuyến di chuyển cho phương tiện trong kịch bản.

Bảng 2.4: Đặc tả use case UC4 – Sinh tuyến đường cho phương tiện

Tên	Sinh tuyến đường cho phương tiện
Tác nhân	Người dùng
Mô tả	Hệ thống sinh tập tuyến đường cho phương tiện dựa trên cách chọn cặp điểm đầu–cuối và lọc các cặp không liên thông.
Tiền điều kiện	Đã có mạng đường hợp lệ.
Luồng chính	(1) Người dùng chọn kiểu phân chia cặp điểm đầu–cuối và số lượng. (2) Hệ thống sinh các cặp theo phân bố tọa độ. (3) Hệ thống kiểm tra tính liên thông của từng cặp. (4) Hệ thống ghi các tuyến hợp lệ ra tệp tuyến đường.
Luồng phụ	(3a) Cặp điểm không có đường đi: hệ thống loại bỏ cặp đó và tiếp tục. (4a) Không còn cặp hợp lệ: hệ thống báo lỗi và yêu cầu cấu hình lại.
Hậu điều kiện	Tệp tuyến đường được tạo, sẵn sàng cho mô phỏng.

2.3.3 Đặc tả use case Chạy mô phỏng thời gian thực

Bảng dưới đây đặc tả use case UC7 – use case cốt lõi của hệ thống, kết nối Python với SUMO và truyền dữ liệu liên tục sang ứng dụng Unity để hiển thị.

Bảng 2.5: Đặc tả use case UC7 – Chạy mô phỏng thời gian thực

Tên	Chạy mô phỏng thời gian thực
Tác nhân	Người dùng, SUMO
Mô tả	Hệ thống chạy mô phỏng và truyền trạng thái từng bước sang ứng dụng hiển thị để cập nhật tức thời.
Tiền điều kiện	Đã có kịch bản mô phỏng hợp lệ; ứng dụng hiển thị đã kết nối.
Luồng chính	(1) Người dùng chọn chế độ thời gian thực và khởi chạy. (2) Hệ thống khởi tạo SUMO và mở các kênh truyền. (3) Mỗi bước, hệ thống lấy trạng thái phương tiện, đèn, người đi bộ. (4) Hệ thống đóng gói và gửi sang ứng dụng hiển thị. (5) Ứng dụng hiển thị cập nhật cảnh. (6) Lặp lại đến khi hết mô phỏng hoặc có lệnh dừng.
Luồng phụ	(3a) Gói tin bị phân mảnh trên đường truyền: phía nhận đệm dữ liệu đến khi đủ một bản tin. (2a) Không kết nối được ứng dụng hiển thị: hệ thống thử lại; nếu thất bại thì dừng.
Hậu điều kiện	Trạng thái mô phỏng được hiển thị theo thời gian; dữ liệu phiên được ghi lại.

2.3.4 Đặc tả use case Tạm dừng và tiếp tục mô phỏng

Bảng dưới đây đặc tả use case UC11 – thao tác điều khiển quá trình chạy mô phỏng, đảm bảo đồng bộ trạng thái giữa SUMO và ứng dụng hiển thị.

Bảng 2.6: Đặc tả use case UC11 – Tạm dừng và tiếp tục mô phỏng

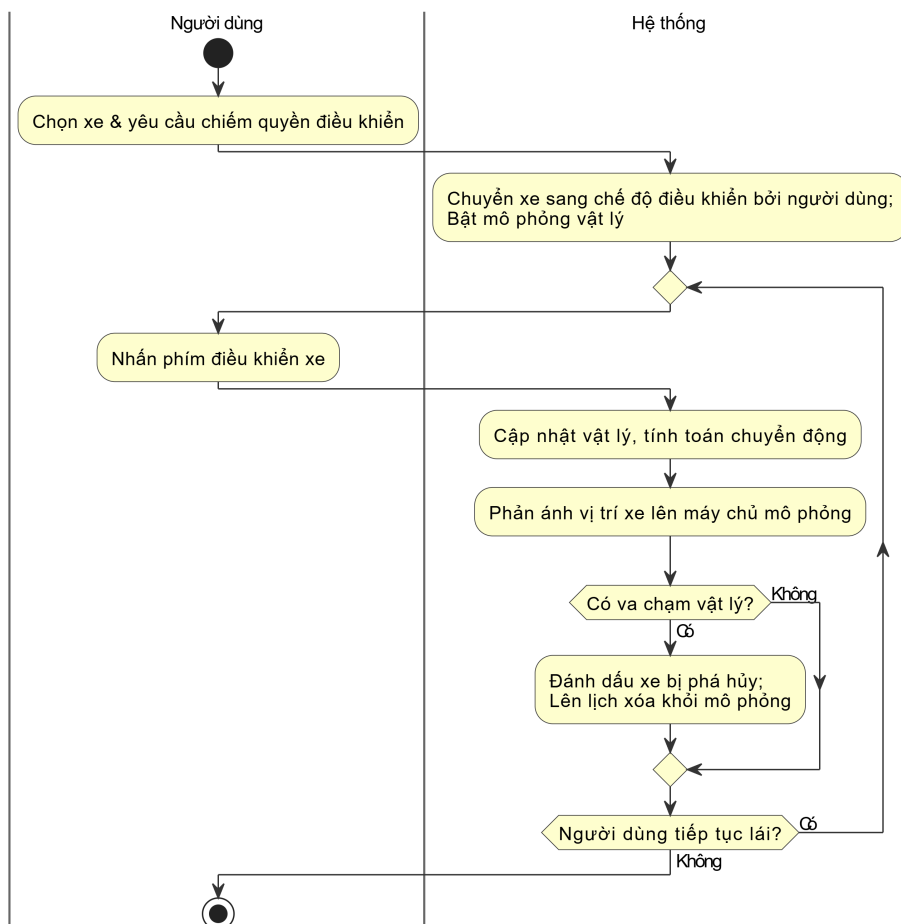
Tên	Tạm dừng và tiếp tục mô phỏng
Tác nhân	Người dùng
Mô tả	Người dùng tạm dừng hoặc tiếp tục quá trình mô phỏng đang chạy để quan sát kỹ một thời điểm.
Tiền điều kiện	Mô phỏng đang chạy ở chế độ thời gian thực.
Luồng chính	(1) Người dùng gửi lệnh tạm dừng. (2) Hệ thống dừng tiến trình bước mô phỏng nhưng giữ nguyên trạng thái. (3) Người dùng gửi lệnh tiếp tục. (4) Hệ thống tiếp tục bước mô phỏng từ trạng thái đã giữ.
Luồng phụ	(2a) Khi tạm dừng, các đối tượng động (kể cả người đi bộ) phải đứng yên, không tiếp tục chuyển động tại chỗ.
Hậu điều kiện	Mô phỏng ở trạng thái tạm dừng hoặc đang chạy theo lệnh.

2.3.5 Đặc tả use case Chiếm quyền điều khiển và lái xe

Bảng dưới đây đặc tả use case UC14. Do use case này có luồng trạng thái phức tạp (chuyển đổi giữa điều khiển tự động và thủ công, xử lý va chạm), biểu đồ hoạt động ở Hình 2.7 mô tả chi tiết các nhánh.

Bảng 2.7: Đặc tả use case UC14 – Chiếm quyền điều khiển và lái xe

Tên	Chiếm quyền điều khiển và lái xe
Tác nhân	Người dùng, SUMO
Mô tả	Người dùng chọn một phương tiện đang do SUMO điều khiển và chuyển nó sang chế độ lái thủ công bằng vật lý.
Tiền điều kiện	Mô phỏng đang chạy; tồn tại phương tiện có thể chiếm quyền.
Luồng chính	(1) Người dùng chọn phương tiện và yêu cầu chiếm quyền. (2) Hệ thống chuyển phương tiện sang chế độ điều khiển bởi người dùng và bật vật lý. (3) Người dùng điều khiển bằng bàn phím. (4) Hệ thống phản ánh vị trí xe sang SUMO để các xe khác phản ứng.
Luồng phụ	(3a) Xe đang lái va chạm với xe khác: hệ thống đánh dấu trạng thái va chạm để xử lý khi trả quyền (xem UC16).
Hậu điều kiện	Phương tiện ở trạng thái do người dùng điều khiển.



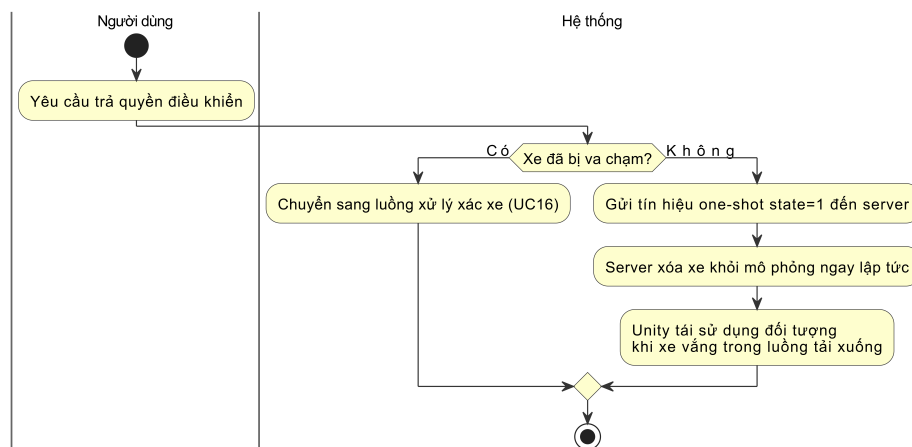
Hình 2.7: Biểu đồ hoạt động use case Chiếm quyền và lái xe

2.3.6 Đặc tả use case Trả quyền điều khiển phương tiện

Bảng dưới đây đặc tả use case UC15. Hình 2.8 mô tả luồng hoạt động của use case này.

Bảng 2.8: Đặc tả use case UC15 – Trả quyền điều khiển phương tiện

Tên	Trả quyền điều khiển phương tiện
Tác nhân	Người dùng, SUMO
Mô tả	Người dùng thả một phương tiện đang lái; hệ thống xóa xe khỏi mô phỏng ngay lập tức và Unity tái sử dụng đối tượng khi xe vắng trong luồng tải xuống.
Tiền điều kiện	Phương tiện đang ở trạng thái do người dùng điều khiển.
Luồng chính	(1) Người dùng yêu cầu trả quyền. (2) Hệ thống gửi tín hiệu one-shot lên server. (3) Server xóa xe khỏi mô phỏng ngay. (4) Unity tái sử dụng đối tượng khi không còn nhận được dữ liệu xe đó trong luồng tải xuống.
Luồng phụ	(1a) Xe ở trạng thái va chạm: chuyển sang xử lý xác xe thay vì xóa ngay (UC16).
Hậu điều kiện	Phương tiện bị xóa khỏi mô phỏng hoặc chuyển sang trạng thái xác xe.



Hình 2.8: Biểu đồ hoạt động use case Trả quyền điều khiển

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng quan trọng.

Hiệu năng. Hệ thống phải duy trì tốc độ khung hình ổn định khi số lượng phương tiện lớn. Để đạt được điều này, ứng dụng hiển thị áp dụng các kỹ thuật như tái sử dụng đối tượng, lọc đối tượng ngoài vùng quan sát và giảm số lệnh vẽ. Việc truyền dữ liệu theo từng bước cũng cần kiểm soát kích thước gói tin để không gây

nghe.

Độ tin cậy. Kênh truyền dữ liệu phải chịu được hiện tượng phân mảnh và gộp gói của giao thức truyền theo luồng, thông qua cơ chế tách khung bản tin. Khi kết thúc, hệ thống phải đóng kết nối và ghi đầy đủ kết quả, tránh mất dữ liệu hay rò rỉ tài nguyên.

Tính mở rộng. Dữ liệu trao đổi giữa phần mô phỏng và phần hiển thị cần được chuẩn hóa theo một định dạng độc lập với nền tảng hiển thị, để có thể thay thế hoặc bổ sung nền tảng hiển thị khác mà không phải thay đổi phần mô phỏng.

Tính dễ dùng. Người dùng cấu hình và khởi chạy mô phỏng thông qua giao diện khởi chạy, không cần thao tác trực tiếp với dòng lệnh phức tạp. Các thao tác quan sát và điều khiển trong khi chạy được cung cấp ngay trên giao diện ba chiều.

Chương này đã khảo sát hiện trạng các công cụ quan sát mô phỏng giao thông, chỉ ra khoảng trống về khả năng hiển thị ba chiều có tương tác theo góc nhìn người tham gia giao thông, và phân tích yêu cầu của hệ thống thông qua mười sáu use case thuộc bốn nhóm chức năng. Các use case quan trọng đã được đặc tả chi tiết kèm biểu đồ hoạt động cho những luồng phức tạp. Các yêu cầu này là cơ sở để lựa chọn công nghệ ở Chương 3 và thiết kế hệ thống ở Chương 4.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã phân tích yêu cầu và mô tả các chức năng của hệ thống. Chương này trình bày các công nghệ và nền tảng được sử dụng để hiện thực các yêu cầu đó, phân tích vai trò của từng thành phần trong kiến trúc tổng thể, và giải thích lý do lựa chọn so với các phương án thay thế. Các công nghệ chính gồm SUMO, TraCI, Python và Unity; ngoài ra chương cũng trình bày cách tích hợp các thành phần này thành một hệ thống mô phỏng–hiển thị thống nhất.

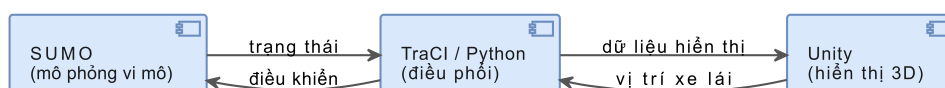
3.1 Tổng quan công nghệ và vai trò trong hệ thống

Bài toán cần giải quyết là hiển thị giao thông trong không gian ba chiều dựa trên dữ liệu mô phỏng, đồng thời cho phép tương tác. Để đạt được mục tiêu này, hệ thống cần đáp ứng đồng thời bốn nhóm yêu cầu: (i) mô phỏng giao thông vi mô theo thời gian, (ii) trích xuất và điều khiển mô phỏng theo từng bước thời gian, (iii) trực quan hóa kết quả dưới dạng ba chiều có khả năng quan sát và tương tác, và (iv) xử lý vật lý cho phương tiện khi người dùng lái thủ công.

Ngăn xếp công nghệ được lựa chọn gồm bốn thành phần chính:

- **SUMO** (Simulation of Urban Mobility): nền tảng mô phỏng giao thông đô thị mã nguồn mở, hỗ trợ mô phỏng vi mô theo từng phương tiện.
- **TraCI** (Traffic Control Interface): giao diện cho phép chương trình bên ngoài kết nối, truy vấn và điều khiển SUMO theo từng bước thời gian.
- **Python**: lớp điều phối chịu trách nhiệm khởi tạo mô phỏng, giao tiếp qua TraCI, chuẩn hóa và truyền dữ liệu.
- **Unity**: môi trường dựng cảnh, hiển thị ba chiều theo thời gian thực và xử lý tương tác, vật lý.

Về mặt kiến trúc, luồng dữ liệu tổng quát có thể mô tả như sau: SUMO mô phỏng giao thông và cung cấp trạng thái đối tượng theo thời gian; Python dùng TraCI lấy trạng thái này theo từng bước và truyền sang Unity theo một định dạng chuẩn; Unity đọc dữ liệu, ánh xạ hệ tọa độ và dựng cảnh. Khi người dùng chiếm quyền điều khiển một phương tiện, dữ liệu còn đi theo chiều ngược lại: Unity gửi vị trí phương tiện do người dùng lái về Python để phản ánh vào SUMO. Hình 3.1 minh họa kiến trúc tổng quát này.



Hình 3.1: Kiến trúc tổng quát hệ thống mô phỏng và hiển thị 3D

3.2 SUMO (Simulation of Urban Mobility)

3.2.1 Giới thiệu

SUMO là công cụ mô phỏng giao thông đô thị mã nguồn mở, được thiết kế để mô phỏng giao thông theo mô hình vi mô [1], [2]. Trong mô hình vi mô, mỗi phương tiện được xem như một thực thể có trạng thái riêng gồm vị trí, vận tốc, gia tốc và hướng di chuyển, tuân theo các quy tắc chuyển động, tuân thủ làn đường và điều khiển giao lộ. Cách tiếp cận này phù hợp khi mục tiêu là tái hiện chuyển động phương tiện liên tục theo thời gian và phục vụ trực quan hóa.

Trong phạm vi đề án, SUMO được sử dụng để tạo ra dữ liệu mô phỏng nhất quán theo thời gian. Thay vì chỉ xuất báo cáo thống kê tổng hợp, hệ thống cần lấy trạng thái chi tiết của từng phương tiện theo từng bước thời gian để hiển thị chuyển động trong Unity.

3.2.2 Lý do lựa chọn SUMO

SUMO được lựa chọn vì các lý do sau:

- **Phù hợp bài toán mô phỏng đô thị:** hỗ trợ mạng đường dạng đô thị nút–cạnh, mô phỏng làn đường, giao lộ và tín hiệu đèn.
- **Khả năng điều khiển theo thời gian:** kết hợp với TraCI cho phép chạy mô phỏng theo từng bước và truy vấn trạng thái tức thời.
- **Dễ tích hợp:** định dạng tệp rõ ràng, hệ sinh thái công cụ phong phú, thuận tiện để Python xử lý và tự động hóa.
- **Mã nguồn mở:** phù hợp với mục tiêu học thuật và triển khai trong khuôn khổ đề án.

3.2.3 Các thành phần cấu hình và dữ liệu trong SUMO

Để chạy một kịch bản, SUMO cần ba nhóm tệp chính: tệp mô tả mạng đường, tệp mô tả luồng/tuyến phương tiện và tệp cấu hình mô phỏng.

Mạng đường. Mạng đường thể hiện hạ tầng giao thông ở dạng đồ thị có hướng gồm nút và cạnh. Mạng có thể bắt đầu từ các tệp rời mô tả nút và cạnh, sau đó biên dịch thành tệp mạng hoàn chỉnh chứa thông tin làn đường, tọa độ và kết nối tại nút giao.

Luồng/tuyến. Luồng phương tiện được mô tả trong tệp tuyến đường, trong đó định nghĩa tuyến, loại phương tiện và thời điểm khởi hành. Khi cần tái hiện giao thông có mật độ thay đổi, cách xây dựng tuyến cần điều chỉnh được lưu lượng và phân bố.

Cấu hình chạy. Tệp cấu hình tập hợp các thành phần trên và chỉ định tham số

mô phỏng như thời gian bắt đầu, kết thúc, độ dài bước thời gian và chế độ xuất dữ liệu. Việc chuẩn hóa tham số giúp các thí nghiệm có thể chạy lại và so sánh.

Bảng 3.1: Đầu vào và đầu ra của SUMO trong hệ thống

Nhóm	Mô tả
Đầu vào	Mạng đường, luồng/tuyến, tệp cấu hình mô phỏng
Đầu ra	Trạng thái theo thời gian: danh sách phương tiện, vị trí, vận tốc, hướng; trạng thái đèn; thông tin làn/cạnh khi cần

3.2.4 Các lựa chọn thay thế và đánh giá

Với bài toán mô phỏng giao thông, có thể xem xét nhiều công cụ khác. Một số công cụ thương mại như Aimsun hoặc VISSIM có khả năng mô phỏng mạnh mẽ nhưng đi kèm hạn chế về giấy phép và chi phí trong khuôn khổ đề án. Ở chiều ngược lại, các công cụ mô phỏng vĩ mô tập trung vào lưu lượng tổng thể, không tối ưu cho việc lấy trạng thái chi tiết của từng phương tiện để hiển thị ba chiều liên tục. Các nền tảng ba chiều phục vụ xe tự hành như CARLA cung cấp mô phỏng cảm biến nhưng yêu cầu tài nguyên lớn hơn nhiều so với mục tiêu hiển thị dòng giao thông dựa trên dữ liệu mô phỏng.

3.3 TraCI (Traffic Control Interface)

3.3.1 Giới thiệu

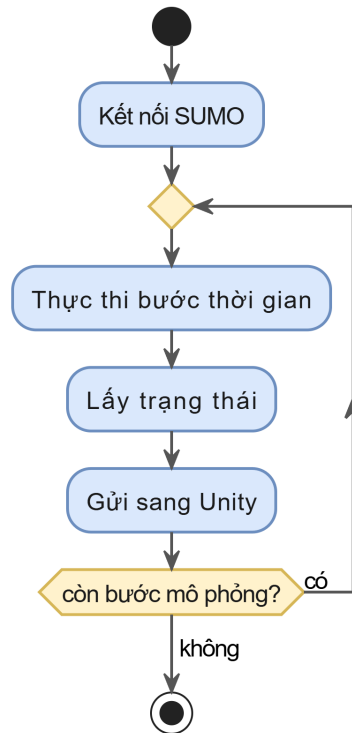
TraCI là giao diện cho phép một chương trình bên ngoài kết nối tới SUMO để điều khiển mô phỏng theo từng bước thời gian [4]. Thay vì để SUMO tự chạy độc lập và chỉ xuất nhật ký sau khi kết thúc, TraCI cho phép hệ thống điều khiển tiến trình mô phỏng theo bước, truy vấn trạng thái đối tượng tại thời điểm đang chạy, thay đổi trạng thái hoặc ra lệnh điều khiển, và đồng bộ dữ liệu với hệ thống hiển thị.

Trong đề án, TraCI là cầu nối quan trọng để Python lấy dữ liệu theo thời gian và cung cấp cho Unity. Nhờ đó, Unity không cần đọc các tệp nhật ký hậu kỳ mà có thể cập nhật chuyển động theo nhịp mô phỏng.

3.3.2 Mô hình hoạt động client-server

Cần lưu ý hệ thống có hai quan hệ máy chủ–máy khách tách biệt, không nên nhầm lẫn. Trong quan hệ với SUMO thông qua TraCI, SUMO là máy chủ còn Python là máy khách. Ngược lại, trong quan hệ truyền dữ liệu với Unity (trình bày ở Mục 3.6.1), Python lại đóng vai trò máy chủ còn Unity là máy khách. Như vậy Python giữ vai trò kép: là máy khách của SUMO để điều khiển mô phỏng, đồng thời là máy chủ của Unity để cung cấp dữ liệu hiển thị.

Xét riêng quan hệ với SUMO: SUMO chạy như một máy chủ, còn Python với vai trò máy khách khởi tạo kết nối, gửi lệnh thực thi bước thời gian, sau đó truy vấn các đối tượng quan tâm như danh sách xe đang tồn tại, vị trí từng xe và trạng thái đèn tín hiệu. Mỗi vòng lặp tương ứng với một bước thời gian, từ đó hình thành luồng dữ liệu theo chuỗi thời gian. Hình 3.2 minh họa vòng lặp này.



Hình 3.2: Vòng lặp điều khiển và thu thập dữ liệu bằng TraCI

3.3.3 Nhóm chức năng TraCI sử dụng trong đồ án

Tùy theo kịch bản và dữ liệu cần hiển thị, TraCI cung cấp nhiều nhóm hàm truy vấn và điều khiển. Trong đồ án, các nhóm thường dùng gồm:

- **Mô phỏng:** điều khiển tiến trình, lấy thời gian mô phỏng và số xe đang tồn tại.
- **Phương tiện:** lấy và cập nhật trạng thái xe gồm tọa độ, tốc độ, hướng; truy vấn tuyến và làn hiện tại. Nhóm này cũng được dùng khi người dùng chiếm quyền điều khiển để phản ánh vị trí xe do người lái vào mô phỏng, và để xóa xe khỏi mô phỏng khi người dùng trả quyền.
- **Đèn tín hiệu:** lấy trạng thái pha đèn của từng nút giao.
- **Cạnh/làn:** lấy thông tin hạ tầng và hình học làn đường phục vụ hiển thị.

3.3.4 Lựa chọn thay thế

Một cách tiếp cận thay thế là chạy SUMO và xuất nhật ký sau mô phỏng, rồi để Unity đọc lại và phát lại. Cách này đơn giản, dễ kiểm tra, nhưng hạn chế khả năng đồng bộ theo thời gian và khó mở rộng sang tình huống cần can thiệp mô phỏng trong lúc chạy, ví dụ khi người dùng lái xe và tác động lên dòng giao thông. Do đó, TraCI phù hợp hơn với mục tiêu kết nối mô phỏng và hiển thị có tương tác. Trên thực tế, đồ án dùng TraCI cho chế độ thời gian thực và dùng cách phát lại từ tệp cho chế độ phát lại.

3.4 Python cho điều phối và xử lý dữ liệu

3.4.1 Vai trò trong hệ thống

Python đóng vai trò lớp điều phối giữa mô phỏng và hiển thị. Ở mức hệ thống, Python chịu trách nhiệm khởi tạo phiên làm việc với SUMO, điều khiển mô phỏng theo từng bước qua TraCI, thu thập dữ liệu cần thiết, chuẩn hóa dữ liệu theo một định dạng trao đổi thống nhất, và truyền dữ liệu sang Unity. So với việc viết toàn bộ lớp điều phối trong Unity, Python mang lại lợi thế về tốc độ phát triển, khả năng thử nghiệm nhanh và thuận tiện cho việc viết kịch bản tự động tạo dữ liệu.

3.4.2 Chuẩn hóa theo định dạng trao đổi độc lập nền tảng

Một mục tiêu quan trọng là tách biệt tầng mô phỏng và thu thập dữ liệu khỏi tầng hiển thị. Vì vậy, Python xuất dữ liệu theo một định dạng trao đổi có thể tái sử dụng bởi nhiều nền tảng đồ họa khác nhau, thống nhất các nội dung tối thiểu như đơn vị đo, hệ tọa độ nguồn, các trường dữ liệu bắt buộc và cấu trúc bản ghi. Nhờ cách thiết kế này, khi thay đổi nền tảng hiển thị, phần Python có thể giữ nguyên, chỉ cần xây dựng một tầng đọc dữ liệu mới ở phía hiển thị.

3.4.3 Tổ chức xử lý theo bước thời gian

Python chạy vòng lặp theo từng bước: mỗi bước tiến mô phỏng một khoảng thời gian, sau đó lấy trạng thái và gửi sang Unity. Trong các bước này có thể áp dụng tối ưu như chỉ gửi các phương tiện đang tồn tại. Để chuyển động mượt trong Unity, hệ thống lựa chọn nội suy ở phía hiển thị thay vì tăng tần suất gửi dữ liệu.

3.4.4 Lựa chọn thay thế

Về mặt kỹ thuật, TraCI cũng hỗ trợ máy khách viết bằng Java hoặc C++. Tuy nhiên, với đồ án ứng dụng, Python là lựa chọn cân bằng giữa tính linh hoạt, khả năng xử lý dữ liệu và thời gian phát triển. Viết trực tiếp lớp giao tiếp trong Unity sẽ giảm một tầng trung gian nhưng làm tăng độ phức tạp và khó tái sử dụng các kịch bản xử lý dữ liệu.

3.5 Unity cho hiển thị 3D và tương tác

3.5.1 Giới thiệu và lý do lựa chọn

Unity là nền tảng phát triển ứng dụng ba chiều thời gian thực, hỗ trợ dựng cảnh, điều khiển camera, chiếu sáng, vật lý và xây dựng giao diện tương tác [5]. Trong đồ án, Unity đảm nhiệm trực quan hóa giao thông ba chiều dựa trên dữ liệu mô phỏng, cho phép người dùng quan sát mật độ xe, dòng xe qua nút giao và hành vi di chuyển theo thời gian. Unity được lựa chọn vì có hệ sinh thái phong phú cho dựng cảnh và xử lý chuyển động, khả năng cập nhật đối tượng theo thời gian thực, hệ thống vật lý sẵn có phục vụ chế độ lái, và phù hợp với yêu cầu triển khai nhanh.

3.5.2 Pipeline hiển thị đối tượng giao thông

Để hiển thị một mô phỏng giao thông, Unity quản lý hai nhóm đối tượng: hạ tầng gồm đường, làn, nút giao; và đối tượng động gồm phương tiện và người đi bộ. Một pipeline hiển thị thường gồm các bước: khởi tạo hạ tầng dựa trên dữ liệu mạng; khởi tạo phương tiện theo danh sách và gán mô hình; và cập nhật vị trí, hướng theo dữ liệu nhận được tại mỗi bước. Khi số lượng xe lớn, hệ thống áp dụng tái sử dụng đối tượng và chỉ hiển thị trong phạm vi quan sát để duy trì hiệu năng.

3.5.3 Hệ thống vật lý của Unity cho chế độ lái

Khi người dùng chiếm quyền điều khiển một phương tiện, phương tiện đó chuyển từ chế độ được cập nhật vị trí theo dữ liệu mô phỏng sang chế độ chuyển động theo vật lý. Unity cung cấp các thành phần cần thiết cho chế độ này. Thành phần thân cứng (Rigidbody) [6] cho phép phương tiện chịu tác động của lực, va chạm và quán tính. Thành phần bộ va chạm bánh xe (Wheel Collider) [7] mô phỏng lực kéo, lực phanh và độ bám của từng bánh, qua đó tái hiện chuyển động lái có tính vật lý thay vì dịch chuyển tức thời. Khi phương tiện không do người dùng điều khiển, hệ thống tắt phần vật lý và chuyển về chế độ cập nhật vị trí theo dữ liệu để bảo đảm hiệu năng. Cơ chế chuyển đổi giữa hai chế độ này là một điểm kỹ thuật quan trọng, được trình bày chi tiết trong Chương 5.

3.5.4 Lựa chọn thay thế

Các nền tảng ba chiều khác như Unreal Engine có khả năng đồ họa mạnh nhưng thường yêu cầu tài nguyên lớn và chu trình phát triển nặng hơn. Các giải pháp trên nền web thuận lợi cho triển khai trực tuyến nhưng cần tự xây dựng nhiều thành phần engine và tối ưu hiệu năng. Vì vậy Unity là lựa chọn phù hợp cho mục tiêu trực quan hóa ba chiều có tương tác trong đồ án.

3.6 Tích hợp SUMO–TraCI/Python–Unity

3.6.1 Phương thức trao đổi dữ liệu

Trong đồ án, dữ liệu được truyền theo thời gian thực bằng giao thức TCP [8]. Ở quan hệ này, Python đóng vai trò máy chủ và phía gửi, còn Unity đóng vai trò máy khách và phía nhận, liên tục đọc gói dữ liệu và cập nhật trạng thái hiển thị. Đây chính là quan hệ máy chủ–máy khách thứ hai đã nêu ở Mục 3.3.2, và ngược chiều với quan hệ giữa Python và SUMO. Cách truyền trực tiếp qua TCP có độ trễ thấp, đồng bộ tốt với tiến trình mô phỏng theo từng bước. Khi người dùng lái xe, một kênh truyền theo chiều ngược lại được dùng để Unity gửi vị trí phương tiện do người lái về phía Python.

Vì TCP là giao thức hướng luồng, một lần nhận có thể chứa một phần hoặc nhiều bản tin gộp lại. Do đó, hệ thống bổ sung cơ chế tách khung bản tin bằng một dấu hiệu kết thúc đặt ở cuối mỗi bản tin; chi tiết được trình bày trong Chương 5.

3.6.2 Định dạng dữ liệu tối thiểu

Để hiển thị chuyển động phương tiện, Unity tối thiểu cần biết định danh xe, vị trí, hướng và vận tốc tại mỗi mốc thời gian. Ngoài ra, để phục vụ phân tích và mở rộng, dữ liệu có thể kèm theo thông tin làn/cạnh hoặc trạng thái đèn. Định dạng này được thiết kế độc lập với engine: các trường phản ánh trạng thái mô phỏng và hình học nguồn, còn các bước chuyển đổi sang hệ trục ba chiều, tỉ lệ hiển thị và nội suy theo khung hình do phía hiển thị đảm nhiệm.

Bảng 3.2: Một số trường dữ liệu trao đổi giữa Python và Unity

Trường dữ liệu	Ý nghĩa
Định danh	Định danh duy nhất của phương tiện trong mô phỏng
Vị trí	Tọa độ của phương tiện trong hệ của SUMO
Hướng	Hướng di chuyển để xoay mô hình ba chiều
Vận tốc	Tốc độ của phương tiện
Trạng thái đèn	Trạng thái đỏ/vàng/xanh của đèn tín hiệu

3.6.3 Đồng bộ thời gian giữa bước mô phỏng và khung hình

SUMO vận hành theo bước thời gian rời rạc với độ dài cố định, trong khi Unity cập nhật theo số khung hình mỗi giây và có thể thay đổi theo cấu hình máy. Nếu Unity chỉ cập nhật vị trí xe đúng tại mỗi bước, chuyển động có thể bị giật khi tốc độ khung hình cao. Có hai cách tiếp cận: cập nhật đúng theo bước, đơn giản và chính xác; hoặc nội suy theo khung hình, trong đó Unity nhận hai trạng thái liên tiếp và nội suy vị trí cho các khung hình ở giữa để chuyển động mượt hơn. Đồ án lựa chọn nội suy theo khung hình ở phía hiển thị nhằm bảo đảm chuyển động mượt và ổn

định.

3.6.4 Kết chương

Chương này đã trình bày các công nghệ chính được sử dụng trong đồ án gồm SUMO, TraCI, Python và Unity, cùng vai trò của từng thành phần trong kiến trúc tổng thể. Chương cũng phân tích cách tích hợp giữa mô phỏng và hiển thị ba chiều, đặc biệt là vấn đề trao đổi dữ liệu hai chiều và đồng bộ thời gian, và giới thiệu hệ thống vật lý của Unity phục vụ chế độ lái. Đây là nền tảng cho phần phân tích thiết kế và triển khai hệ thống ở Chương 4.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày các công nghệ nền tảng. Chương này trình bày cách thiết kế và hiện thực hệ thống dựa trên các công nghệ đó, gồm: kiến trúc tổng thể, thiết kế các mô-đun chính, máy trạng thái điều khiển phương tiện, hai luồng dữ liệu giữa mô phỏng và hiển thị, cùng kết quả thực nghiệm và đánh giá. Nội dung tập trung vào những điểm phản ánh đúng hiện thực của hệ thống thay vì mô tả lý thuyết chung.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

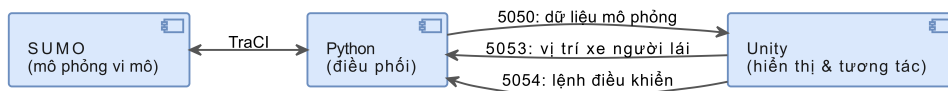
Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp mô hình mô phỏng–hiển thị, trong đó phần mô phỏng giao thông chạy trên SUMO (điều khiển bằng TraCI trong Python) và phần hiển thị, tương tác chạy trên Unity. Hai phần giao tiếp qua kết nối TCP theo cơ chế truyền thông điệp dạng JSON, có dấu hiệu kết thúc gói tin <END> để tách thông điệp trên luồng TCP.

Kiến trúc này được chọn vì các lý do: tách biệt trách nhiệm giữa mô phỏng và hiển thị; dễ mở rộng thêm loại đối tượng hoặc chỉ số thống kê mà ít ảnh hưởng phần còn lại; và tương thích với công cụ, do SUMO đã có TraCI hỗ trợ điều khiển theo bước, còn Unity thuận lợi cho hiển thị thời gian thực và xử lý vật lý. Ở mức khái niệm, có thể ánh xạ theo mô hình Model–View–Controller: phần Model là trạng thái mô phỏng trong SUMO cùng các tệp mạng và tuyến; phần View là các đối tượng và mô-đun dựng cảnh trong Unity; phần Controller là lớp giao tiếp TCP và bộ xử lý lệnh điều khiển.

4.1.2 Kiến trúc tổng quan

Hệ thống gồm ba tầng: tầng mô phỏng (Python kết hợp SUMO/TraCI) khởi tạo mạng, chạy mô phỏng theo bước, trích xuất trạng thái và ghi kết quả; tầng tích hợp (TCP cùng định dạng JSON) đóng gói, tách khung và truyền thông điệp; tầng hiển thị (Unity) nhận dữ liệu, dựng và cập nhật đối tượng, cung cấp giao diện điều khiển và xử lý tương tác lái xe.

Điểm đáng chú ý trong hiện thực là hệ thống dùng ba kênh TCP tách theo mục đích và hai luồng dữ liệu ngược chiều, như minh họa ở Hình 4.1. Kênh chính truyền dữ liệu mô phỏng từ Python sang Unity; kênh cập nhật nhận dữ liệu phương tiện do người dùng lái từ Unity về Python; kênh điều khiển nhận các lệnh tạm dừng, tiếp tục, kết thúc và cập nhật tham số.



Hình 4.1: Kiến trúc chi tiết với ba kênh TCP và hai luồng dữ liệu

4.1.3 Thiết kế các mô-đun chính

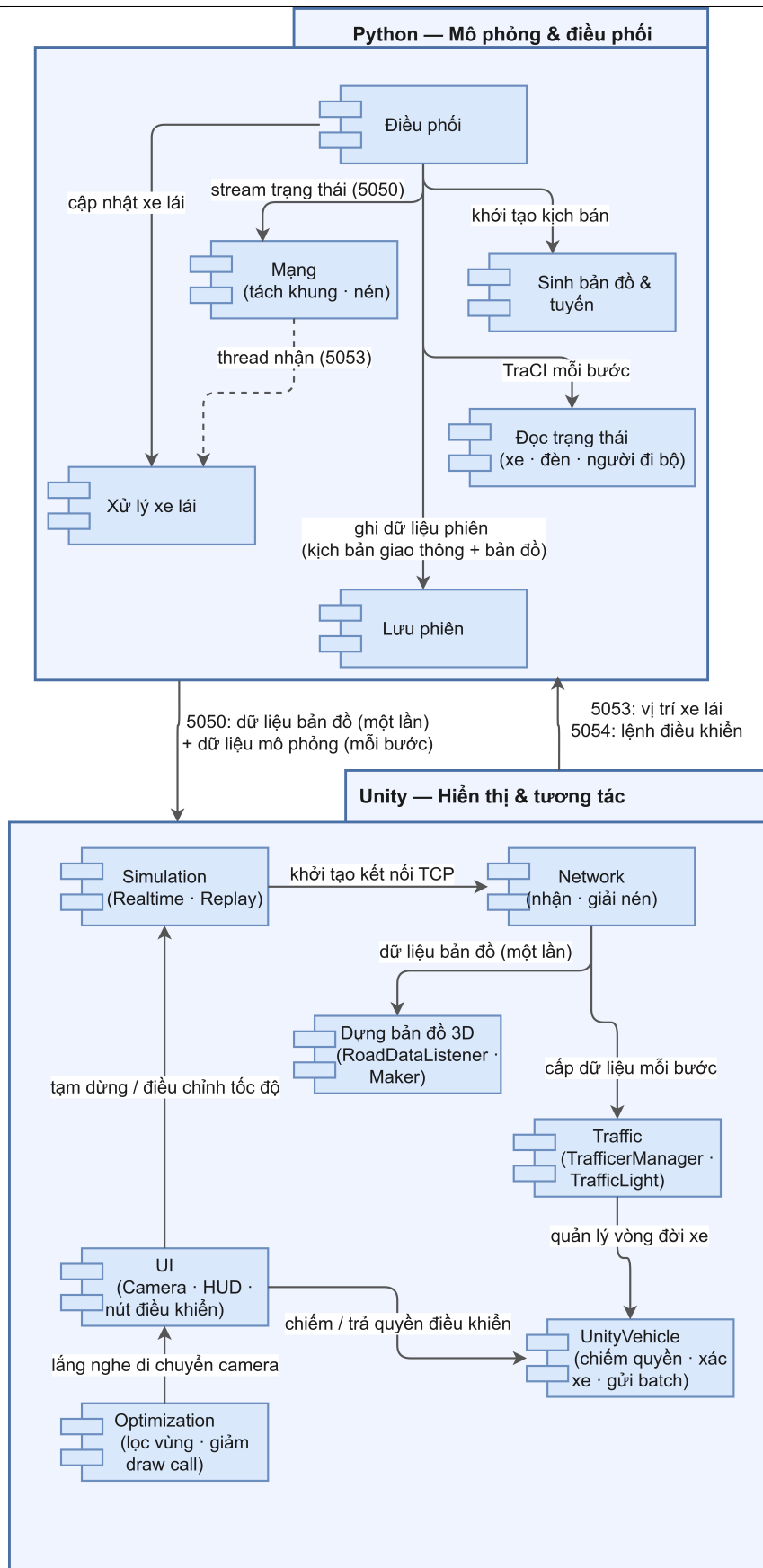
Phía Python đảm nhiệm mô phỏng và điều phối, gồm sáu nhóm mô-đun chính: mô-đun *Điều phối* khởi tạo các kênh TCP, chạy vòng lặp mô phỏng và điều phối toàn bộ luồng xử lý; mô-đun *Mạng* đóng gói, tách khung thông điệp và nén dữ liệu tải xuống; nhóm mô-đun *Đọc trạng thái* trích xuất dữ liệu xe, đèn tín hiệu và người đi bộ từ TraCI sang định dạng JSON chuẩn; mô-đun *Xử lý xe lái* nhận lô dữ liệu từ Unity và phản ánh vào SUMO (mirror/freeze/hủy); nhóm mô-đun *Sinh bản đồ & tuyến* tạo mạng đường và lịch trình di chuyển cho kịch bản mô phỏng; và mô-đun *Lưu phiên* ghi kết quả mỗi lần chạy vào thư mục phiên gồm nhật ký hành trình, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo bước.

Phía Unity đảm nhiệm hiển thị và tương tác, gồm bảy nhóm mô-đun chính trình bày trong Bảng 4.1. Trong đó, nhóm mô-đun *Dựng bản đồ 3D* nhận dữ liệu mạng đường từ Python một lần khi khởi chạy và dựng lưới đa giác cho mặt đường, nút giao, vạch sang đường và công trình, tạo nên cảnh ba chiều nền cho toàn bộ phiên mô phỏng.

Bảng 4.1: Các nhóm mô-đun chính phía Unity

Nhóm mô-đun	Nhiệm vụ
Mạng (Network)	Kết nối TCP, nhận dữ liệu mạng đường và dữ liệu mô phỏng.
Dựng bản đồ 3D (Road)	Nhận dữ liệu mạng đường một lần khi khởi chạy; dựng Mesh mặt đường, nút giao, vạch sang đường và công trình.
Giao thông (Traffic)	Quản lý vòng đời và cập nhật phương tiện, đèn tín hiệu, người đi bộ; nội suy chuyển động.
Phương tiện người lái (UnityVehicle)	Chiếm quyền, lái bằng vật lý, gửi dữ liệu xe lên server, xử lý va chạm.
Tối ưu (Optimization)	Lọc đối tượng theo vùng quan sát, giảm số lệnh vẽ.
Giao diện (UI)	Điều khiển camera, tạm dừng, tốc độ, nút chiếm/trả quyền.
Mô phỏng (Simulation)	Điều phối chế độ thời gian thực và chế độ phát lại.

Cách phân chia mô-đun và quan hệ phụ thuộc giữa hai phía được minh họa ở Hình 4.2.



Hình 4.2: Sơ đồ gói các nhóm mô-đun hai phía và quan hệ phụ thuộc

4.2 Thiết kế chi tiết

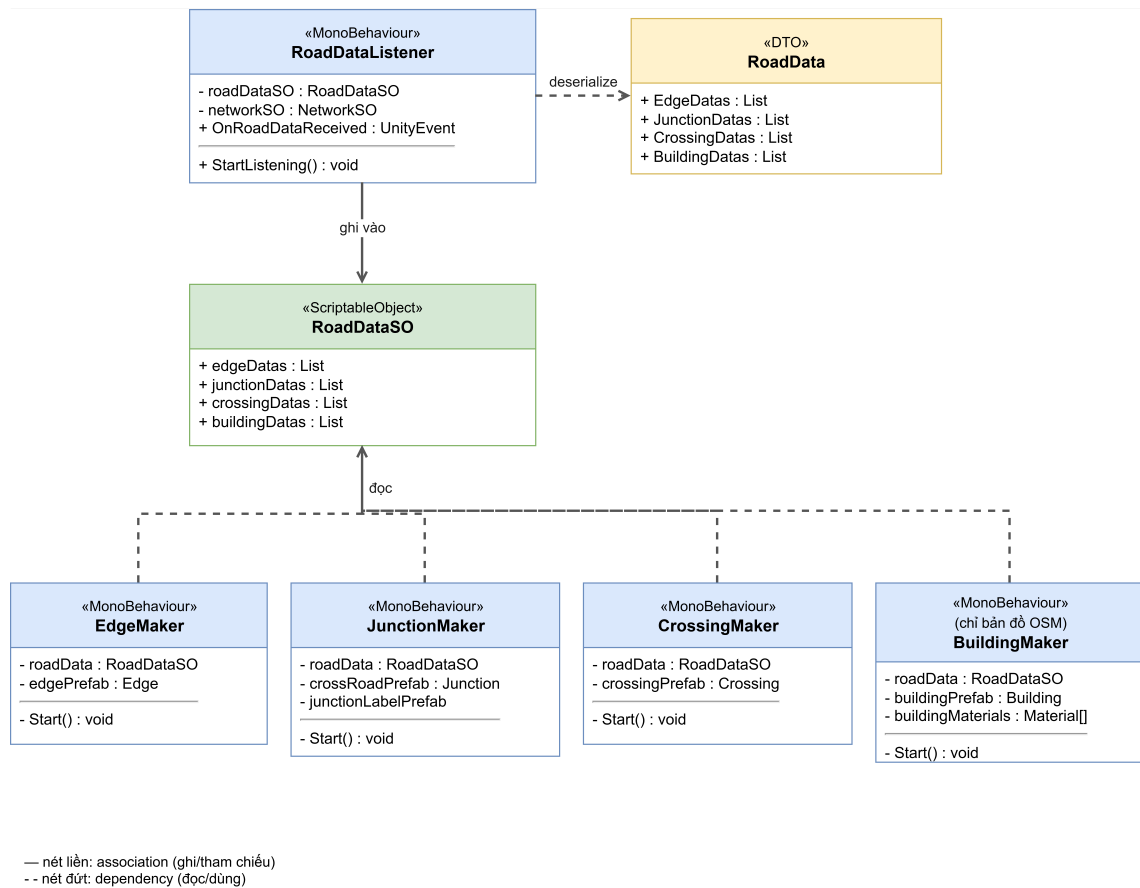
4.2.1 Thiết kế module dựng bản đồ 3D

Module dựng bản đồ 3D chịu trách nhiệm xây dựng cảnh ba chiều nền — gồm mặt đường, nút giao, vạch sang đường và công trình — từ dữ liệu mạng đường của SUMO. Module này chỉ chạy *một lần* khi khởi chạy, trước khi vòng lặp mô phỏng bắt đầu, và hoạt động hoàn toàn tách biệt với vòng lặp đó.

Phía Python. Khi Unity gửi yêu cầu "RoadDataRequest" qua cổng 5050, `render_map.py` gọi tuần tự bốn nguồn dữ liệu: lớp `EdgeReader` (`edgeType0.py`) trích xuất đa tuyến trung tâm và chiều rộng từng làn đường từ tệp `.net.xml` của SUMO; lớp `CrossRoadReader` (`crossRoad.py`) trích xuất hình dạng đa giác nút giao; lớp `CrossingReader` (`crossing.py`) trích xuất vạch sang đường; và một hàm riêng đọc đa giác công trình từ tệp `OpenStreetMap` nếu bản đồ có nguồn gốc OSM. Toàn bộ dữ liệu được đóng gói thành một gói JSON và gửi về Unity qua cổng 5050; đồng thời ghi ra tệp `road_data.json` trong thư mục phiên để phục vụ kiểm tra.

Phía Unity. Hình 4.3 trình bày cấu trúc lớp module phía Unity. `RoadDataListener` nhận gói JSON, giải tuần tự hóa vào lớp trung gian `RoadData`, rồi chép dữ liệu sang `RoadDataSO` — một `ScriptableObject` đóng vai trò kho dữ liệu dùng chung cho bốn lớp `Maker`. Mỗi `Maker` đọc `RoadDataSO` trong `Start()` và dựng Mesh cho một loại đối tượng địa lý: `EdgeMaker` cho mặt đường, `JunctionMaker` cho nút giao, `CrossingMaker` cho vạch sang đường, và `BuildingMaker` cho công trình (chỉ hoạt động với bản đồ OSM). Sau khi bốn `Maker` hoàn tất, cảnh ba chiều nền sẵn sàng để vòng lặp mô phỏng cập nhật phương tiện lên trên. Chi tiết thuật toán dựng Mesh được trình bày trong Chương 5.

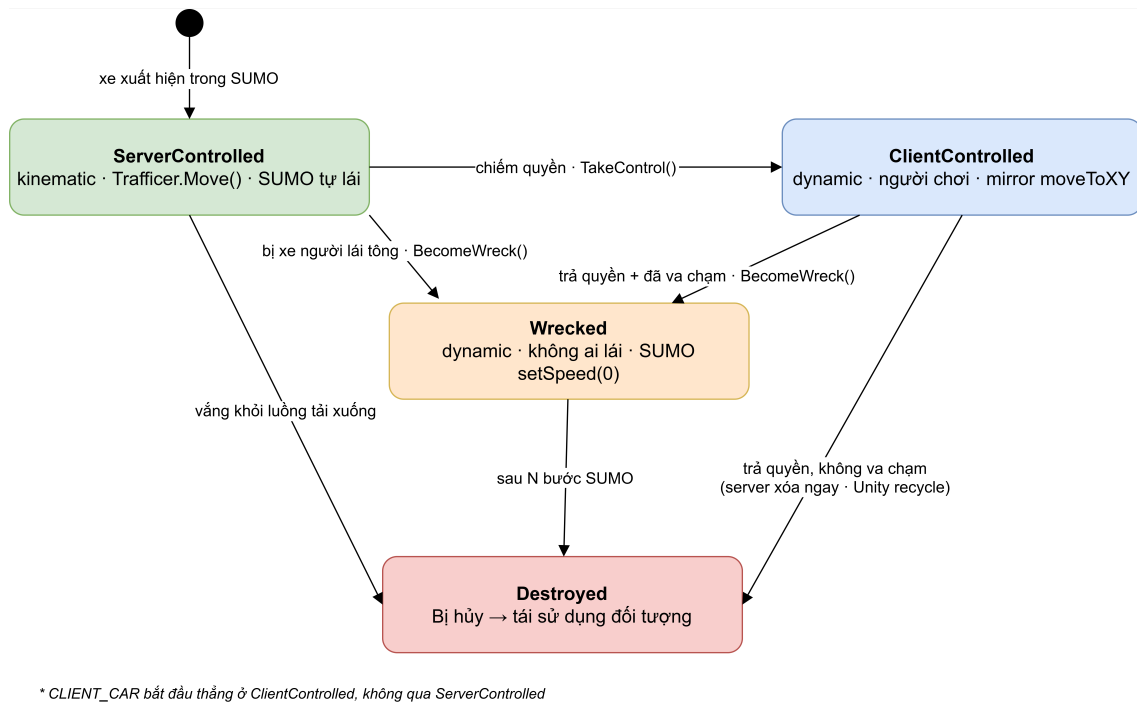
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.3: Sơ đồ lớp module dựng bản đồ 3D phía Unity

4.2.2 Máy trạng thái điều khiển phương tiện

Điểm thiết kế cốt lõi của phần tương tác là một máy trạng thái xác định quyền điều khiển và hành vi của mỗi phương tiện. Mỗi phương tiện luôn ở một trong bốn trạng thái: được SUMO điều khiển, được người dùng điều khiển, trở thành xác xe sau va chạm, hoặc bị hủy. Quyền điều khiển và việc bật/tắt vật lý được quyết định hoàn toàn bởi trạng thái này, thay vì dựa vào sự hiện diện của thành phần. Hình 4.4 mô tả máy trạng thái.



Hình 4.4: Máy trạng thái điều khiển phương tiện

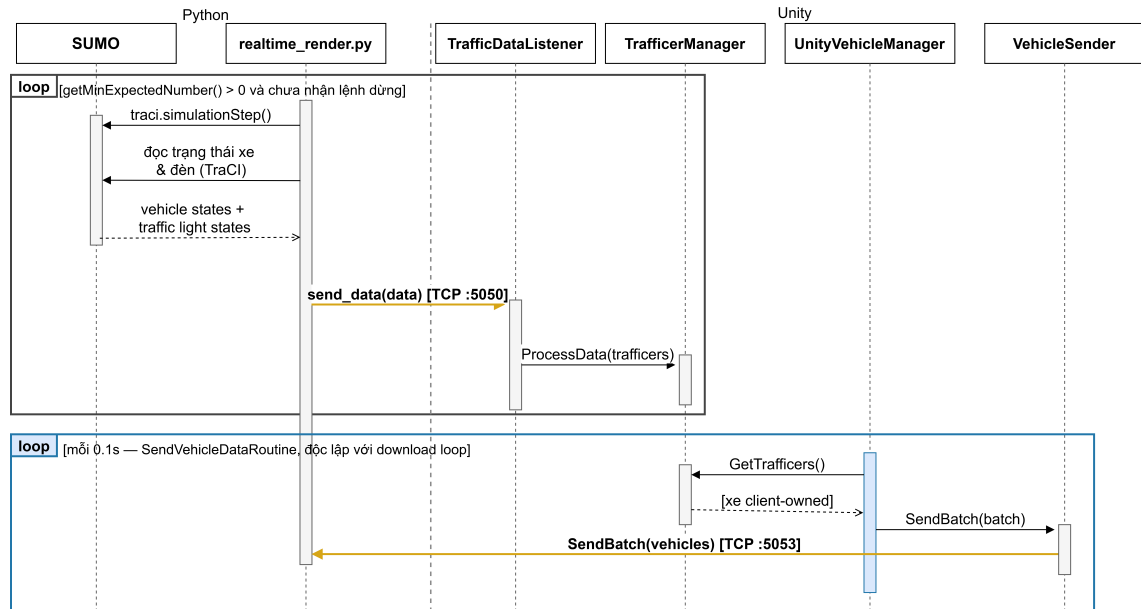
Khi một phương tiện do SUMO điều khiển, vị trí của nó được cập nhật theo dữ liệu mô phỏng và phần vật lý được tắt. Khi người dùng chiếm quyền, hệ thống bật vật lý để người dùng lái; vị trí xe được phản ánh ngược về SUMO để các xe khác phản ứng. Khi một xe đang do người dùng điều khiển hoặc một xác xe va chạm với xe do SUMO điều khiển, xe bị tông chuyển thành xác xe, phía SUMO đặt tốc độ về 0 để tạo vật cản gây ùn ứ trong khi vẫn đồng bộ vị trí từ Unity, và tự bị hủy sau một số bước định trước. Khi trả quyền mà xe không va chạm, server nhận tín hiệu một lần và xóa xe khỏi mô phỏng ngay lập tức; phía Unity tái sử dụng đối tượng khi xe vắng khỏi luồng tải xuống. Trạng thái bị hủy cũng đưa phương tiện về bộ nhớ tái sử dụng theo cùng cơ chế.

4.2.3 Hai luồng dữ liệu giữa mô phỏng và hiển thị

Hệ thống có hai luồng dữ liệu ngược chiều chạy hoàn toàn độc lập với nhau. Luồng tải xuống truyền trạng thái mô phỏng từ Python sang Unity sau mỗi bước mô phỏng, gồm chỉ số bước hiện tại, trạng thái đèn tín hiệu và danh sách đối tượng giao thông; trong đó mỗi đối tượng chứa định danh, loại, vị trí, hướng và một số cờ trạng thái. Để giảm băng thông, dữ liệu tải xuống được nén theo thuật toán raw-deflate và mã hóa base64 trước khi gửi; Unity nhận biết gói nén qua tiền tố GZ : và giải nén bằng DeflateStream. Luồng tải lên chạy độc lập theo coroutine mỗi 0,1 giây, không đồng bộ với nhịp tải xuống: UnityVehicleManager lấy danh sách xe do người dùng điều khiển từ TrafficerManager, đóng gói thành một lô duy nhất gồm định danh, vị trí, hướng, tốc độ và trạng thái tồn tại của từng xe,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

rồi chuyển cho `VehicleSender` gửi qua cổng 5053. Phía Python dùng luồng tải lên để phản ánh xe người lái vào SUMO và đối soát danh sách phương tiện đang được quản lý. Hình 4.5 minh họa hai luồng này qua hai khung vòng lặp riêng biệt: khung đen là luồng tải xuống, khung xanh là luồng tải lên.



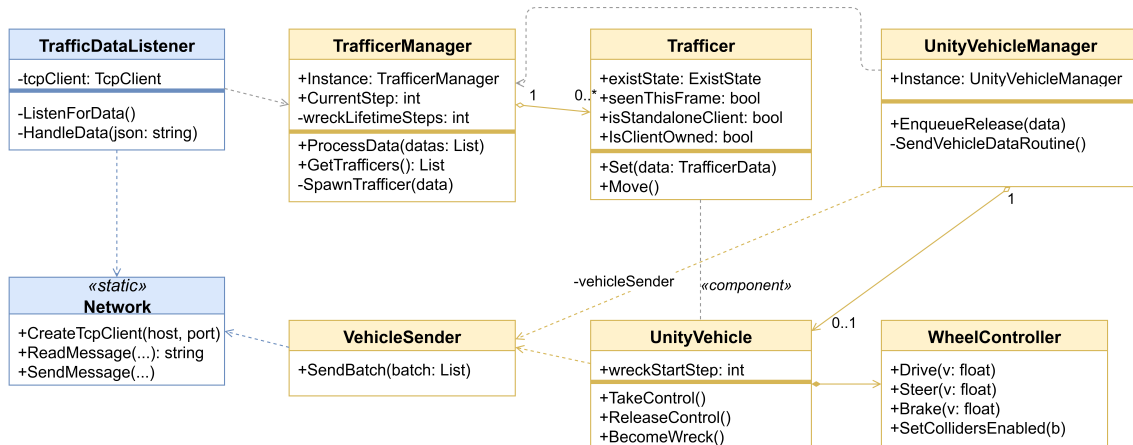
Hình 4.5: Trình tự trao đổi thông điệp giữa mô phỏng và hiển thị trong một bước

4.2.4 Thiết kế module giao thông

Trong phạm vi báo cáo, một số lớp chủ đạo của module giao thông được trình bày như sau.

- `TrafficManager` (Unity): quản lý vòng đời phương tiện theo dữ liệu tải xuống, tạo, cập nhật và thu hồi đối tượng; đếm tập trung số bước để hủy xác xe nhằm tránh lỗi khi đối tượng bị tạm ẩn.
- `UnityVehicle` (Unity): xử lý chiếm quyền, trả quyền và chuyển xe thành xác xe; bật/tắt vật lý theo trạng thái; thu thập dữ liệu xe để gửi lên.
- `WheelController` (Unity): điều khiển bốn bánh xe gồm tăng tốc, đánh lái và phanh khi xe ở chế độ lái.
- `VehicleSender` (Unity): gửi dữ liệu phương tiện theo lô lên cổng 5053; giữ lô mới nhất để truyền, bỏ qua các lô cũ hơn nếu kịp nhận thêm.
- `UnityVehicleManager` (Unity): điều phối luồng tải lên; coroutine của lớp này cứ 0,1 giây lấy danh sách xe `IsClientOwned` từ `TrafficManager`, đóng gói thành lô và chuyển cho `VehicleSender`; ngoài ra gửi tín hiệu one-shot trả quyền (`state = 1`) để server xóa xe ngay.

Quan hệ giữa các lớp chủ đạo phía Unity được minh họa ở Hình 4.6.

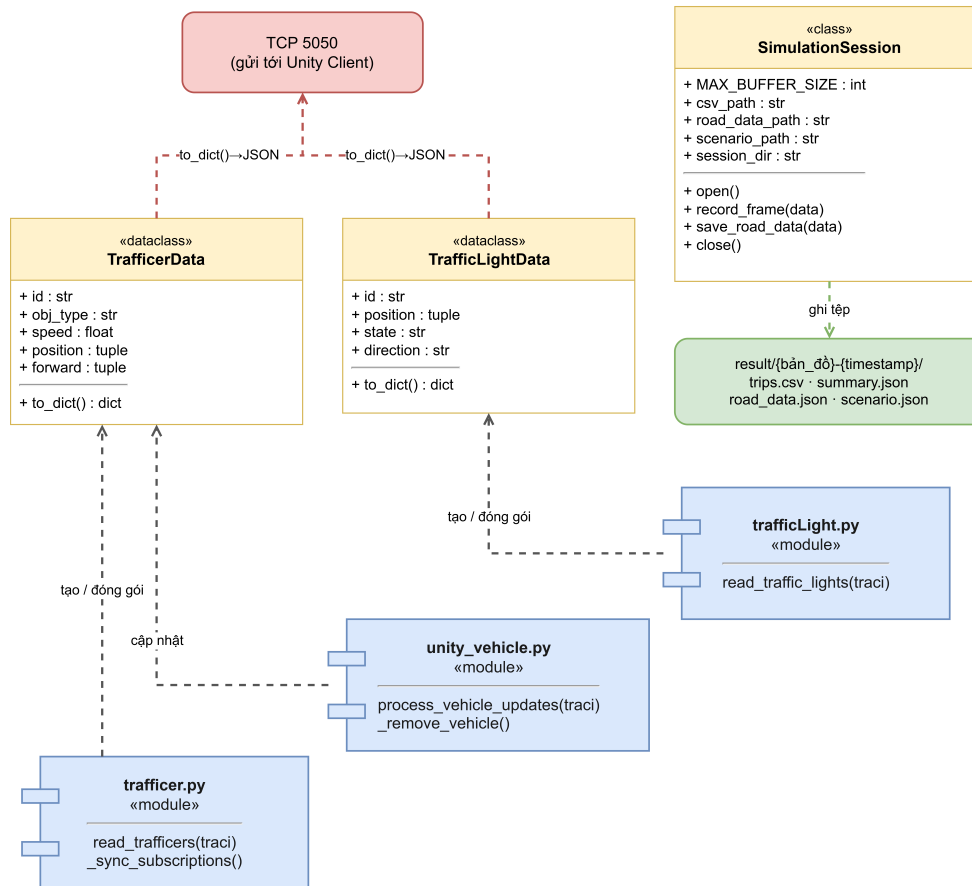


Hình 4.6: Sơ đồ lớp các lớp chủ đạo phía Unity (nhóm quản lý phương tiện và điều khiển lái).

Ngoài nhóm quản lý phương tiện, module giao thông phía Unity còn có nhóm đèn tín hiệu với lớp `TrafficLight` (bật/tắt đèn đỏ–vàng–xanh theo trạng thái nhận được) và `TrafficLightManager` (tạo, cập nhật và tái sử dụng đối tượng đèn). Cấu trúc hai lớp này tương tự nhóm phương tiện nhưng không đưa vào sơ đồ để giữ hình gọn.

P phía Python server, các lớp và mô-đun giao thông được tổ chức thành ba nhóm: tầng dữ liệu gồm `TrafficerData` và `TrafficLightData` đóng gói trạng thái từng đối tượng; tầng đọc trạng thái gồm các mô-đun `trafficier.py`, `unity_vehicle.py` và `trafficLight.py` gọi `TraCI` để trích xuất dữ liệu mỗi bước; và lớp `SimulationSession` gom toàn bộ kết quả của một lần chạy vào thư mục phiên `result/{bản_đồ}-{thời_điểm}/`, mở tệp `scenario.json` ở chế độ `streaming` và ghi từng khung trạng thái qua `record_frame()`, tự đẩy đệm khi đạt ngưỡng 10 MB. Hình 4.7 minh họa quan hệ giữa các thành phần này.

Module giao thông phía Python Server



Hộp vàng: class / dataclass Python
 Hộp xanh: module Python (function-level)
 Hộp xanh lá: tệp/thư mục đầu ra
 -- nét đứt: dependency (tạo / dùng / gửi / ghi)

Hình 4.7: Sơ đồ lớp và mô-đun giao thông phía Python server

4.2.5 Lưu trữ dữ liệu

Hệ thống không dùng cơ sở dữ liệu truyền thống mà lưu dữ liệu dưới dạng tệp. Dữ liệu cấu hình và mạng mô phỏng là các tệp của SUMO. Dữ liệu kết quả của mỗi lần chạy được gom vào một thư mục phiên đặt theo tên bản đồ và thời điểm chạy, gồm nhật ký hành trình của xe, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo từng bước. Cách lưu này phù hợp với mục tiêu mô phỏng, hiển thị và thu thập thống kê theo từng lần chạy, đồng thời cho phép phát lại. Cấu trúc một thư mục phiên tiêu biểu được minh họa ở Hình 4.8.

```

result/
└─ HoanKiem-2026-06-13-09-30-05/
    ├── trips.csv           — nhật ký hành trình của xe
    ├── summary.json       — bản tóm tắt lần chạy
    ├── road_data.json     — dữ liệu mạng đường (nút, cạnh, vạch qua đường)
    └─ scenario.json       — chuỗi trạng thái mô phỏng theo từng bước

* trips.csv đổi thành trips-ped.csv khi kịch bản có người đi bộ
  
```

Hình 4.8: Cấu trúc một thư mục phiên tiêu biểu, đặt tên theo bản đồ và thời điểm chạy. Tập `trips.csv` đổi thành `trips-ped.csv` khi kịch bản có người đi bộ.

4.2.6 Thiết kế giao diện người dùng

Khác với các ứng dụng hướng biểu mẫu, giao diện của hệ thống được thiết kế xoay quanh việc *quan sát và tương tác với cảnh ba chiều*, nên thành phần trung tâm không phải là các ô nhập liệu mà là khung nhìn (camera) cùng một lớp điều khiển gọn nhẹ phủ lên trên. Giao diện được tổ chức theo nguyên tắc: ưu tiên không gian hiển thị cho cảnh, các bảng điều khiển chỉ hiện khi cần và có thể ẩn đi; thao tác chính dùng chuột và bàn phím; và các nút mang tính ngữ cảnh, chỉ xuất hiện khi thực sự áp dụng được cho đối tượng đang chọn.

Chế độ hiển thị. Hệ thống có hai chế độ với bố cục giao diện riêng: chế độ thời gian thực (hiển thị mô phỏng đang chạy, kèm các lệnh điều khiển quá trình chạy) và chế độ phát lại (đọc lại một phiên đã lưu). Trước khi vào cảnh, người dùng chọn chế độ, bản đồ và tham số mô phỏng ở bước cấu hình khởi chạy.

Camera và góc nhìn. Camera hỗ trợ hai góc nhìn nhằm phục vụ cả nhu cầu giám sát tổng thể lẫn trải nghiệm của người tham gia giao thông:

- **Góc nhìn tự do:** camera bay tự do trên cao, di chuyển bằng phím (WASD) và xoay bằng chuột, với tốc độ di chuyển và độ nhạy chuột điều chỉnh được. Đây là góc nhìn để quan sát toàn cảnh mạng đường và dòng giao thông.
- **Góc nhìn gắn vào phương tiện:** camera gắn vào một phương tiện cụ thể để nhìn theo nó. Người dùng chọn phương tiện bằng cách trỏ và bấm (point-and-select), hoặc chọn ngẫu nhiên một phương tiện đang được hiển thị trong danh sách. Đây là góc nhìn tạo nên trải nghiệm theo người tham gia giao thông và là tiền đề cho thao tác chiếm quyền điều khiển.

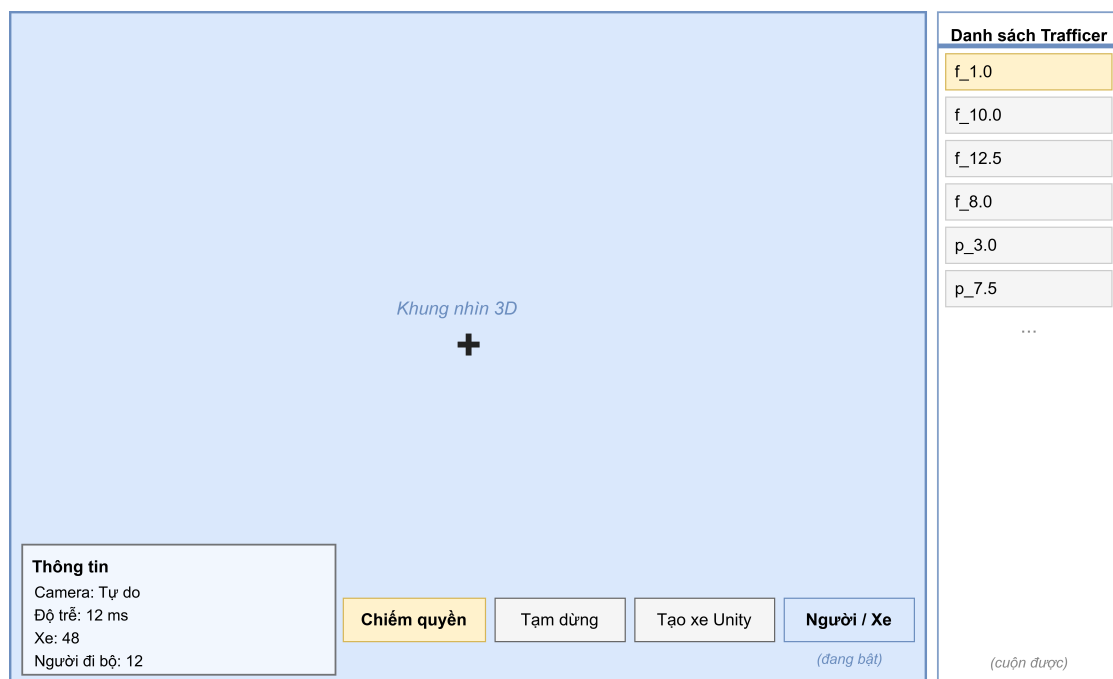
Trạng thái camera hiện hành (tự do hay đang gắn vào phương tiện nào) được hiển thị thường trực để người dùng định hướng.

Lớp điều khiển và bảng thông tin (HUD). Phủ trên khung nhìn là một lớp giao diện gọn nhẹ. Góc dưới bên trái là bảng thông tin hiển thị trạng thái camera (tự do

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

hay đang gắn vào phương tiện nào) cùng các chỉ số tổng quan của phiên chạy: độ trễ đầu–cuối, số phương tiện và số người đi bộ. Độ trễ đầu–cuối được đo bằng cách server Python đánh nhãn thời gian vào mỗi gói dữ liệu ngay trước khi gửi; phía Unity tính hiệu số giữa thời điểm nhận và nhãn đó để ra độ trễ của bước vừa hoàn tất. Vì cả hai tiến trình chạy trên cùng một máy nên đồng hồ hệ thống là thống nhất và số đo phản ánh chính xác chi phí xử lý và truyền dữ liệu qua TCP nội bộ. Góc dưới bên phải là hàng nút điều khiển chính gồm: nút *chiếm/trả quyền điều khiển* mang tính ngữ cảnh (chỉ hiện khi camera đang gắn vào một phương tiện có thể lái, và tự đổi nhãn giữa “Chiếm quyền” và “Trả quyền” theo trạng thái xe); nút *tạm dừng/tiếp tục* mô phỏng; nút *Tạo xe Unity* để sinh một phương tiện riêng do người dùng điều khiển (và đổi thành “Hủy xe Unity” để loại bỏ nó); và nút *Người/Xe* để ẩn/hiện danh sách các đối tượng giao thông (phương tiện và người đi bộ) đang có trong cảnh theo mã định danh, chọn một mục trong danh sách sẽ gắn camera vào đúng đối tượng tương ứng. Hình 4.9 minh họa bố cục giao diện trong cảnh.

Ngoài hàng nút chính, các thiết lập ít dùng hơn được đặt trong các bảng tùy chọn mở bằng phím ESC, gồm: điều chỉnh tốc độ mô phỏng, tốc độ và độ nhạy của camera, tùy chọn lọc theo vùng quan sát phục vụ tối ưu hiển thị, và lệnh kết thúc phiên chạy. Khi một bảng tùy chọn mở, con trỏ chuột được mở khóa để thao tác và được khóa lại khi đóng để tiếp tục điều khiển camera; ngoài ra bảng hướng dẫn phím điều khiển cũng có thể bật/tắt khi cần.



Hình 4.9: Bố cục giao diện người dùng trong cảnh ba chiều

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.2 liệt kê các công cụ chính được sử dụng trong dự án.

Bảng 4.2: Danh sách thư viện và công cụ sử dụng

Mục đích	Công cụ	Phiên bản
Mô phỏng giao thông vi mô	Eclipse SUMO	1.27.0
Điều khiển mô phỏng	TraCI (Python)	đi kèm SUMO
Ngôn ngữ điều phối	Python	3.14+
Hiển thị 3D và tương tác	Unity Editor	6000.3.9f1
Phân tích JSON trong Unity	Newtonsoft.Json	3.2.x
Môi trường phát triển	Visual Studio Code	—
Quản lý phiên bản	Git	—
Hệ điều hành thử nghiệm	Windows	10/11

4.3.2 Kết quả đạt được và thống kê mã nguồn

Sản phẩm chính gồm: bộ sinh mạng và tuyến cho SUMO (từ lưới mê cung hoặc từ bản đồ OpenStreetMap); ứng dụng Unity hiển thị mô phỏng ba chiều có tương tác lái xe; và bộ lưu trữ phiên cùng nhật ký kết quả. Bảng 4.3 thống kê nhanh quy mô mã nguồn do nhóm tự viết, không tính thư viện bên thứ ba.

Bảng 4.3: Thống kê quy mô mã nguồn

Thành phần	Số tệp	Số dòng
Python (mô phỏng và điều phối)	39	5081
Unity (mã nguồn C#)	106	7501

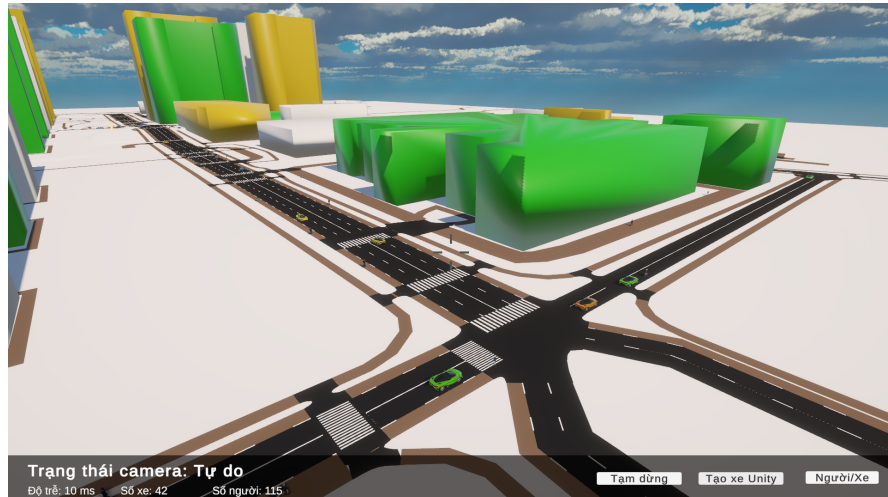
4.3.3 Minh họa các chức năng chính

Các chức năng chính đã hiện thực gồm: sinh mạng đường từ lưới mê cung hoặc bản đồ thực tế; chạy mô phỏng và đồng bộ trạng thái sang Unity theo thời gian thực; hiển thị phương tiện, đèn tín hiệu và người đi bộ trong không gian ba chiều; chiếm quyền điều khiển và lái xe thủ công; xử lý va chạm sinh xác xe và trả quyền xóa xe khỏi mô phỏng; cùng các thao tác điều khiển quá trình chạy. Các hình minh họa dưới đây thể hiện một số chức năng tiêu biểu.

4.4 Kiểm thử

Do hệ thống có tính tích hợp cao giữa SUMO và Unity, việc kiểm thử được thực hiện theo hai hướng: kiểm thử chức năng thủ công theo use case và kiểm thử hộp đen cho các mô-đun xử lý dữ liệu và giao thức. Bảng 4.4 liệt kê một số trường hợp kiểm thử tiêu biểu.

Trong điều kiện chạy thử nghiệm, các trường hợp trên đạt yêu cầu. Hiện tượng



Hình 4.10: Minh họa cảnh giao thông ba chiều trong Unity: góc nhìn từ trên cao

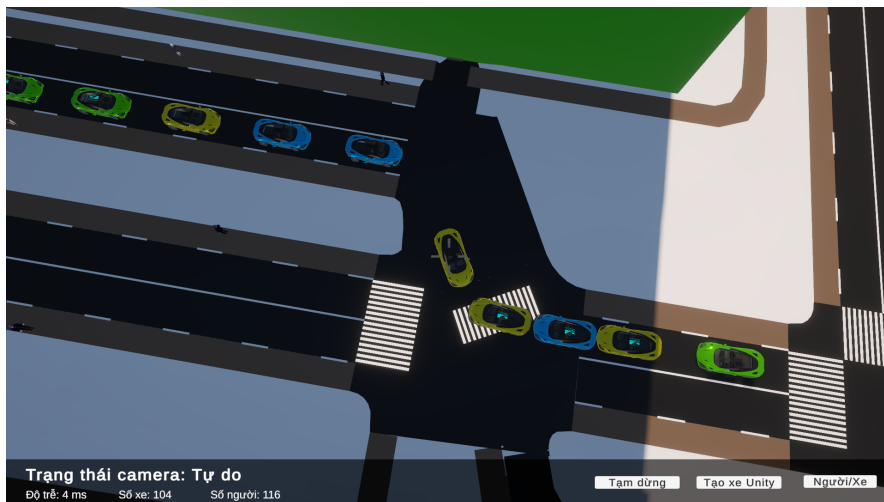


(a) Quan sát xe tự hành, nút “Chiếm quyền” hiện ra



(b) Đang lái xe (camera gắn CLIENT_CAR)

Hình 4.11: Minh họa chế độ lái xe thủ công: từ quan sát đến chiếm quyền điều khiển



Hình 4.12: Minh họa xử lý va chạm: xác xe nằm lại gây ùn ứ dòng giao thông

Bảng 4.4: Một số trường hợp kiểm thử tiêu biểu

Mã	Mục tiêu	Dữ liệu vào / Kỳ vọng
TC01	Tách khung thông điệp TCP	Gửi hai thông điệp liên tiếp trong một lần gửi; phía nhận tách đúng hai thông điệp.
TC02	Nhận dữ liệu mạng đường một lần	Unity yêu cầu dữ liệu mạng; nhận đủ và dựng cảnh đúng.
TC03	Hiển thị trạng thái đèn	Mô phỏng trả về chuỗi trạng thái đèn; Unity hiển thị đúng đỏ/vàng/xanh.
TC04	Chiếm quyền và lái xe	Người dùng chiếm một xe; xe chuyển sang lái bằng vật lý, các xe khác phản ứng theo.
TC05	Va chạm sinh xác xe	Xe người lái tông xe của mô phỏng; xe bị tông thành xác xe và biến mất sau N bước.
TC06	Trả quyền điều khiển	Thả xe không va chạm: xe bị xóa khỏi mô phỏng ngay, Unity tái sử dụng đối tượng; thả xe đã va chạm: xe chuyển sang xác xe (TC05).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lệch đồng bộ chủ yếu xuất hiện khi tốc độ mô phỏng đặt quá cao; khi điều chỉnh tốc độ hợp lý, quan sát ổn định hơn.

4.5 Thực nghiệm và đánh giá

4.5.1 Môi trường và kịch bản thực nghiệm

Hệ thống được thử nghiệm trên máy cá nhân chạy Windows với hai tiến trình: tiến trình Python kết hợp SUMO và tiến trình Unity. Hai nhóm kịch bản được sử dụng: nhóm kịch bản sinh từ lưới mê cung với kích thước tăng dần nhằm khảo sát khả năng chịu tải theo số lượng đối tượng; và nhóm kịch bản từ bản đồ thực tế OpenStreetMap nhằm kiểm chứng khả năng dựng cảnh và hiển thị trên dữ liệu thật.

4.5.2 Phương pháp đánh giá

Việc đánh giá tập trung vào ba khía cạnh. Thứ nhất là tính đúng đắn của hiển thị, được kiểm chứng bằng cách đối chiếu cảnh ba chiều trong Unity với giao diện hai chiều của SUMO trên cùng một kịch bản. Thứ hai là hiệu năng hiển thị, đo bằng tốc độ khung hình khi số lượng phương tiện thay đổi. Thứ ba là tính ổn định của tương tác, quan sát qua các thao tác chiếm quyền, lái xe, va chạm và trả quyền.

4.5.3 Kết quả

Bảng 4.5 ghi nhận kết quả hiệu năng trên máy bàn (Intel Core i5-14400F, NVIDIA RTX 4060, 32 GB RAM DDR4) với Unity giới hạn ở 120 FPS. Ba kịch bản mê cung có kích thước lần lượt 32×32 , 64×64 và 128×128 ô, mỗi kịch bản chạy khoảng 100 phương tiện và 100 người đi bộ; kịch bản OSM chạy khoảng 50 phương tiện và 50 người đi bộ.

Bảng 4.5: Kết quả hiệu năng theo quy mô kịch bản (máy bàn)

Kịch bản	Xe / Người đi bộ	FPS TB	FPS thấp nhất	Độ trễ TB (ms)
Mê cung nhỏ (32×32)	100 / 100	119.5	52.9	6.2
Mê cung vừa (64×64)	100 / 100	112.1	40.6	8.3
Mê cung lớn (128×128)	100 / 100	100.1	11.9	13.0
Bản đồ OSM	50 / 50	119.8	93.3	6.2

Bảng 4.6 đối chiếu hiệu năng kịch bản OSM trên máy tính xách tay (Dell Inspiron 5620, Intel Core i5-1240P, NVIDIA MX570, 16 GB RAM DDR4).

Bảng 4.6: Kết quả hiệu năng kịch bản OSM trên máy tính xách tay

Kịch bản	Xe / Người đi bộ	FPS TB	FPS thấp nhất	Độ trễ TB (ms)
Bản đồ OSM	50 / 50	81.5	40.8	11.8

Kết quả cho thấy trên máy bàn, hệ thống duy trì FPS trung bình sát giới hạn 120 với kịch bản mê cung nhỏ (119,5 FPS) và bản đồ OSM (119,8 FPS). Khi quy

mô tăng lên mê cung vừa và lớn, FPS trung bình giảm lần lượt xuống 112,1 và 100,1, trong khi độ trễ truyền thông tăng từ 6,2 ms lên 8,3 và 13,0 ms do SUMO cần nhiều thời gian tính toán hơn mỗi bước mô phỏng trên mạng đường lớn hơn. Bản đồ OSM với 50 phương tiện và 50 người đi bộ đạt FPS thấp nhất là 93,3, ổn định hơn đáng kể so với mê cung 128×128 (FPS thấp nhất 11,9) vốn xuất hiện các đỉnh tải ngấn trong giai đoạn khởi tạo cảnh. Trên máy tính xách tay phổ thông (i5-1240P, MX570), kịch bản OSM đạt 81,5 FPS trung bình, cho thấy hệ thống có thể vận hành ổn định với các kịch bản quy mô thông thường mà không đòi hỏi phần cứng chuyên dụng. Nhìn chung, kiến trúc pipeline đủ đáp ứng yêu cầu hiển thị thời gian thực trên cả hai lớp thiết bị.

4.5.4 Triển khai

Trong phạm vi đồ án, hệ thống chạy theo mô hình một máy và một người dùng. Quy trình chạy gồm các bước: chuẩn bị kịch bản và khởi chạy tiến trình Python kết hợp SUMO thông qua giao diện khởi chạy; sau đó chạy ứng dụng Unity để kết nối, nhận dữ liệu và hiển thị. Ba kênh TCP được dùng tương ứng cho dữ liệu mô phỏng, dữ liệu phương tiện do người dùng lái và lệnh điều khiển. Kết quả mỗi lần chạy được lưu cục bộ theo từng phiên để phục vụ phân tích và phát lại.

Chương này đã trình bày thiết kế kiến trúc ba tầng với ba kênh TCP và hai luồng dữ liệu, máy trạng thái điều khiển phương tiện, thiết kế các mô-đun và lớp chủ đạo, cùng phương pháp kiểm thử, thực nghiệm và triển khai. Các giải pháp kỹ thuật nổi bật phát sinh trong quá trình thiết kế được trình bày chi tiết trong Chương 5.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này trình bày các giải pháp kỹ thuật và đóng góp nổi bật của đề án trong quá trình xây dựng hệ thống. Mỗi giải pháp được trình bày theo ba phần: vấn đề đặt ra, giải pháp, và kết quả đạt được. Các nội dung về kiến trúc và thiết kế tổng thể đã trình bày ở Chương 4; chương này tập trung vào những điểm mang tính giải quyết vấn đề và đóng gói thành giải pháp có thể tái sử dụng.

5.1 Dựng bản đồ ba chiều chính xác từ SUMO và OpenStreetMap

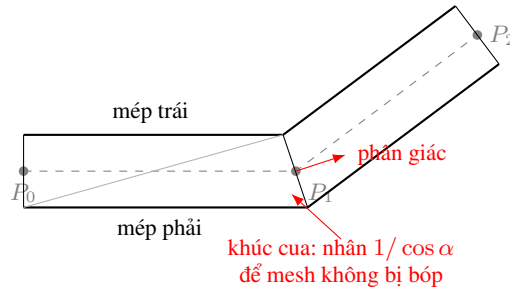
5.1.1 Vấn đề

Yêu cầu cốt lõi của hệ thống là dựng lại mạng đường trong không gian ba chiều bám sát dữ liệu nguồn, cho cả mạng sinh tự động lẫn bản đồ thực tế từ OpenStreetMap. Dữ liệu mạng của SUMO mô tả mỗi làn đường bằng một đường gấp khúc và mỗi nút giao bằng một đa giác hình dạng bất kỳ. Việc chuyển các mô tả này thành lưới đa giác ba chiều phát sinh nhiều khó khăn: dải mặt đường phải bám đúng đường gấp khúc và không bị bóp méo tại khúc cua; đa giác nút giao có thể lõm hoặc thậm chí tự cắt khi SUMO gộp nhiều nút sát nhau; và mặt đường của các đoạn kề nhau phải khớp nhau để xe có thể di chuyển liên tục mà không vướng.

5.1.2 Giải pháp

Đề án xây dựng hai thuật toán dựng hình riêng trong Unity cho hai loại đối tượng là đoạn đường và nút giao.

Thuật toán dựng đoạn đường. Mỗi làn được dựng thành một dải lưới bám theo đường gấp khúc của nó. Tại mỗi đỉnh của đường gấp khúc, hệ thống tính một vector vuông góc với hướng đi để sinh ra hai đỉnh lưới ở hai mép làn. Ở các đỉnh giữa nằm trên khúc cua, vector vuông góc được lấy theo phân giác của hai đoạn kề và độ rộng được nhân thêm một hệ số theo góc cua, nhờ đó dải mặt đường không bị bóp lại ở chỗ gấp khúc. Trên cơ sở đó, hệ thống vẽ thêm vạch phân làn liền hoặc đứt đoạn và vạch biên. Để xe có thể di chuyển trong lòng đường, mỗi đoạn được sinh một tấm sàn va chạm phẳng bao phủ toàn bộ bề mặt đoạn đó. Hình 5.1 minh họa cách dựng dải mặt đường và hiệu chỉnh tại khúc cua; Giải thuật 1 tóm tắt bước sinh đỉnh lưới.



Hình 5.1: Dựng dải mặt đường tám đường gấp khúc và hiệu chỉnh tại khúc cua (bản nháp)

Giải thuật 1: Sinh đỉnh lưới cho một lần đường

Input: Đường gấp khúc $P_0, \dots, P_{n-1} \in \mathbb{R}^3$; bề rộng làn w

Output: Hai dãy đỉnh mép $\{L_i\}, \{R_i\}$ và tập tam giác T

$e_i \leftarrow \frac{P_{i+1} - P_i}{\|P_{i+1} - P_i\|}, \quad i = 0, \dots, n-2; //$ vector đơn vị mỗi đoạn

for $i \leftarrow 0$ **to** $n-1$ **do**

$d_i \leftarrow \begin{cases} e_0, & i = 0 \\ e_{n-2}, & i = n-1 \\ \frac{e_{i-1} + e_i}{\|e_{i-1} + e_i\|}, & 0 < i < n-1 \end{cases}; \quad //$ hướng (phân giác

tại đỉnh giữa)

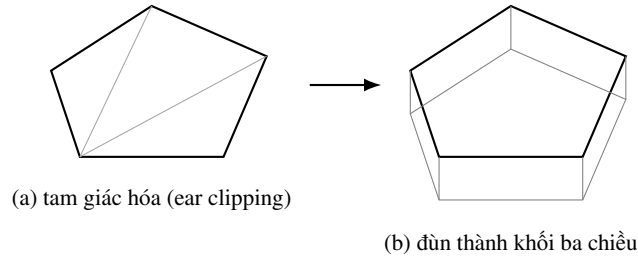
$r_i \leftarrow \frac{d_i \times (-\hat{y})}{\|d_i \times (-\hat{y})\|}; \quad //$ pháp tuyến ngang

$\delta_i \leftarrow \begin{cases} w/2, & i \in \{0, n-1\} \\ \frac{w}{2 \cos \alpha_i}, & 0 < i < n-1 \end{cases} \quad \text{với } \cos \alpha_i = \sqrt{\frac{1+e_{i-1} \cdot e_i}{2}};$

$L_i \leftarrow P_i + \delta_i r_i, \quad R_i \leftarrow P_i - \delta_i r_i;$

$T \leftarrow \bigcup_{i=0}^{n-2} \{(L_i, R_i, L_{i+1}), (R_i, R_{i+1}, L_{i+1})\}; //$ mỗi đoạn $\rightarrow$ 2 tam giác

Thuật toán dựng nút giao. Đa giác nút giao được tam giác hóa bằng kỹ thuật cắt tai (ear clipping) chiếu trên mặt phẳng ngang, nhờ đó giữ được hình dạng lõm của nút. Trong trường hợp đa giác tự cắt do SUMO gộp nhiều nút sát nhau khiến cắt tai không hoàn tất, hệ thống tự chuyển sang phương án bao lồi (convex hull) để bảo đảm mặt nút vẫn được phủ kín thay vì bị rách. Mặt trên sau khi tam giác hóa được đùn xuống một đoạn để tạo thành khối ba chiều có mặt trên, mặt dưới và các mặt bên. Khối nút được gán một bộ va chạm dạng hộp bao để hệ vật lý hoạt động ổn định. Hình 5.2 minh họa quá trình tam giác hóa và đùn khối; Giải thuật 2 tóm tắt các bước.



Hình 5.2: Tam giác hóa đa giác nút giao rồi đùn thành khối ba chiều (bản nháp)

Giải thuật 2: Dựng khối nút giao ba chiều

Input: Đa giác nút giao $V = (v_0, \dots, v_{m-1})$; vector độ cao $\vec{h}$

Output: Tập tam giác lưới khối ba chiều $\mathcal{T}$

$A(V) \leftarrow \frac{1}{2} \sum_{i=0}^{m-1} (x_i y_{i+1} - x_{i+1} y_i)$; // diện tích có dấu, chỉ số mod m

if $A(V) > 0$ **then**

$V \leftarrow (v_{m-1}, \dots, v_0)$; // đảo về thứ tự cùng chiều kim đồng hồ

$T \leftarrow \text{EarClip}(V)$; // tam giác hóa, giữ hình lõm

if $|T| < m - 2$ **then**

$V \leftarrow \text{ConvexHull}(V)$; // đa giác tự cắt $\rightarrow$ bao lồi

$T \leftarrow \text{EarClip}(V)$;

$T_{\text{trên}} \leftarrow T$;

$T_{\text{dưới}} \leftarrow \{ (c + \vec{h}, b + \vec{h}, a + \vec{h}) : (a, b, c) \in T \}$; // dời $\vec{h}$, đảo pháp tuyến

$T_{\text{bên}} \leftarrow \bigcup_{i=0}^{m-1} \{ (v_i, v_{i+1}, v_{i+1} + \vec{h}), (v_i, v_{i+1} + \vec{h}, v_i + \vec{h}) \}$;

$\mathcal{T} \leftarrow T_{\text{trên}} \cup T_{\text{dưới}} \cup T_{\text{bên}}$;

Hai thuật toán này dùng chung cho cả mạng sinh tự động và mạng nhập từ OpenStreetMap, do dữ liệu sau khi chuyển đổi đều ở cùng một định dạng mạng của SUMO. Nhờ đó, bản đồ thực tế cũng được dựng ba chiều bằng cùng một quy trình.

5.1.3 Kết quả đạt được

Hệ thống dựng được mạng đường ba chiều bám sát dữ liệu nguồn cho cả mạng sinh tự động và bản đồ thực tế từ OpenStreetMap. Mặt đường liền lạc tại khúc cua, nút giao được phủ kín kể cả với hình dạng phức tạp, và bề mặt va chạm cho phép xe di chuyển cùng đổi làn liên tục. Đây là nền tảng để hiển thị giao thông và để người dùng lái xe trong cảnh.

5.2 Tạo phương tiện của người dùng và chiếm quyền điều khiển

5.2.1 Vấn đề

Để mang lại góc nhìn của người tham gia giao thông, hệ thống cần cho phép người dùng điều khiển trực tiếp một phương tiện trong cảnh, gồm hai tình huống: tạo một phương tiện riêng của người dùng, và giành quyền điều khiển một phương tiện vốn do SUMO điều khiển. Khó khăn nằm ở chỗ một phương tiện do SUMO điều khiển được cập nhật vị trí theo dữ liệu mô phỏng và không phản ứng vật lý; muốn lái thủ công thì phải chuyển nó sang chuyển động theo vật lý, đồng thời phản ánh vị trí mới về SUMO để các xe khác nhận biết.

5.2.2 Giải pháp

Đồ án thiết kế một máy trạng thái xác định quyền điều khiển của mỗi phương tiện, trong đó quyền điều khiển và việc bật hay tắt vật lý được quyết định bởi trạng thái chứ không dựa vào sự hiện diện của thành phần. Khi người dùng chiếm quyền, phương tiện chuyển sang trạng thái do người dùng điều khiển: hệ thống bật thân cứng (Rigidbody) và các bộ va chạm bánh xe (Wheel Collider) để xe chuyển động theo vật lý, đồng thời ngừng cập nhật vị trí theo dữ liệu mô phỏng. Vị trí của xe do người dùng lái được gửi ngược về phía SUMO để cập nhật vào mô phỏng, giúp các phương tiện khác phản ứng theo. Khi không còn do người dùng điều khiển, phương tiện tắt phần vật lý và trở lại chế độ cập nhật theo dữ liệu để bảo đảm hiệu năng. Cách bật tắt vật lý theo trạng thái như vậy được gọi là cơ chế lai, chỉ trả chi phí vật lý cho đúng phương tiện đang được lái. Bên cạnh đó, hệ thống cho phép sinh một phương tiện riêng của người dùng, là phương tiện luôn do người dùng điều khiển và không bị SUMO chiếm lại.

5.2.3 Kết quả đạt được

Người dùng có thể chiếm quyền một phương tiện bất kỳ đang chạy trong mô phỏng và lái nó bằng bàn phím với chuyển động có tính vật lý, hoặc sinh một phương tiện riêng để lái. Các phương tiện do SUMO điều khiển phản ứng với phương tiện người lái nhờ vị trí được phản ánh ngược về mô phỏng. Đây là tính năng tạo nên trải nghiệm theo góc nhìn người tham gia giao thông mà các công cụ quan sát thông thường không có.

5.3 Hệ thống va chạm giả lập tai nạn

5.3.1 Vấn đề

Khi người dùng lái xe trong dòng giao thông, một tình huống tự nhiên là va chạm với phương tiện khác. Hệ thống cần tái hiện hậu quả của va chạm một cách hợp lý: phương tiện bị tông phải dừng lại như một chiếc xe gặp tai nạn, gây ùn ứ

cho các phương tiện phía sau, và sau một thời gian thì được dọn khỏi hiện trường. Thách thức là phương tiện bị tông vốn do SUMO điều khiển và không phản ứng vật lý, nên cần được chuyển trạng thái ngay tại thời điểm va chạm; ngoài ra việc dọn xe phải đáng tin cậy ngay cả khi đối tượng tạm thời bị ẩn khỏi cảnh.

5.3.2 Giải pháp

Hệ thống phát hiện va chạm giữa phương tiện do người dùng điều khiển (hoặc xác xe) với phương tiện do SUMO điều khiển. Phương tiện bị tông được chuyển sang trạng thái xác xe: nó được bật vật lý để bị xô đẩy theo va chạm, đồng thời phía SUMO đặt tốc độ về 0 và tiếp tục đồng bộ vị trí từ Unity, tạo vật cản gây ùn ứ cho dòng xe phía sau. Mỗi xác xe được dọn sau một số bước mô phỏng định trước. Để việc dọn xe đáng tin cậy, bộ đếm số bước được quản lý tập trung theo bước mô phỏng thay vì đặt trên từng đối tượng, nhờ đó không bị sai lệch khi đối tượng tạm bị ẩn để tối ưu hiển thị. Khi đủ số bước, phương tiện được chuyển sang trạng thái bị hủy và được thu hồi.

5.3.3 Kết quả đạt được

Hệ thống tái hiện được tình huống tai nạn: xe bị tông dừng lại gây ùn ứ giống va chạm thực tế, sau đó tự được dọn khỏi hiện trường. Tính năng này làm phong phú các tình huống mà người dùng có thể quan sát và thử nghiệm, và là hệ quả tự nhiên của khả năng lái xe tương tác.

5.4 Lưu trữ kịch bản và phát lại

5.4.1 Vấn đề

Để phục vụ phân tích, đối chiếu và trình bày, hệ thống cần lưu lại diễn biến của mỗi lần chạy và cho phép xem lại mà không phải chạy lại mô phỏng. Nếu mỗi loại dữ liệu được ghi rời rạc thì khó truy vết theo từng lần chạy; ngoài ra, mô phỏng dài sinh khối lượng dữ liệu lớn nên không thể giữ toàn bộ trong bộ nhớ trước khi ghi.

5.4.2 Giải pháp

Đồ án xây dựng cơ chế lưu trữ theo phiên, gom toàn bộ kết quả của một lần chạy vào một thư mục đặt theo tên bản đồ và thời điểm chạy, gồm nhật ký hành trình của xe, bản tóm tắt, dữ liệu mạng đường và chuỗi trạng thái theo từng bước. Chuỗi trạng thái được ghi theo cơ chế luồng với một vùng đệm: dữ liệu được ghi dần ra tệp khi vùng đệm đầy, nhờ đó không cần giữ toàn bộ trong bộ nhớ. Trên cơ sở chuỗi trạng thái đã lưu, hệ thống cung cấp chế độ phát lại đọc lại dữ liệu và tái hiện diễn biến trong Unity. Đáng chú ý, chế độ thời gian thực cũng ghi lại chuỗi trạng thái theo cùng định dạng, nên một lần chạy thời gian thực cũng có thể được phát lại về sau.

5.4.3 Kết quả đạt được

Mỗi lần chạy được lưu trọn vẹn trong một thư mục phiên, thuận tiện cho truy vết và so sánh. Người dùng có thể phát lại một kịch bản đã lưu mà không cần chạy lại mô phỏng, phục vụ tốt cho việc phân tích và trình bày kết quả.

5.5 Giao tiếp thời gian thực cây bằng tách khung bản tin

5.5.1 Vấn đề

Unity nhận dữ liệu trạng thái theo từng bước từ phía Python qua TCP. Vì TCP là giao thức hướng luồng, một lần nhận có thể chỉ chứa một phần bản tin hoặc chứa nhiều bản tin gộp lại, dẫn tới lỗi khi phân tích dữ liệu. Hệ thống cần một cơ chế xác định ranh giới bản tin ổn định, đồng thời hỗ trợ cả bản tin dữ liệu lẫn bản tin điều khiển.

5.5.2 Giải pháp

Mỗi bản tin được gửi kèm một dấu hiệu kết thúc đặt ở cuối. Phía nhận duy trì một vùng đệm cho mỗi kết nối, đọc thêm dữ liệu cho tới khi gặp dấu hiệu kết thúc, tách ra đúng một bản tin hoàn chỉnh và giữ lại phần dư cho lần sau. Cách này xử lý đúng bản chất luồng của TCP, tránh cả hai lỗi bản tin bị cắt và nhiều bản tin dính nhau. Trên nền giao thức này, hệ thống truyền song song bản tin dữ liệu trạng thái và bản tin điều khiển như tạm dừng, tiếp tục, kết thúc và cập nhật tham số.

5.5.3 Kết quả đạt được

Kênh giao tiếp giữa Python và Unity hoạt động ổn định trong suốt quá trình mô phỏng thời gian thực, giảm lỗi phân tích dữ liệu và hỗ trợ điều khiển quá trình chạy một cách an toàn.

5.6 Tối ưu hiệu năng hiển thị quy mô lớn

5.6.1 Vấn đề

Hệ thống cần hiển thị đồng thời nhiều phương tiện trong khi vẫn duy trì tốc độ khung hình ổn định. Việc liên tục tạo và hủy đối tượng, cũng như vẽ toàn bộ đối tượng kể cả ngoài tầm quan sát, làm giảm hiệu năng đáng kể khi số lượng phương tiện tăng.

5.6.2 Giải pháp

Hệ thống áp dụng một số kỹ thuật tối ưu. Thứ nhất là tái sử dụng đối tượng: phương tiện rời mạng không bị hủy mà được thu hồi để dùng lại cho phương tiện mới, giảm chi phí tạo và hủy. Thứ hai là lọc theo vùng quan sát: chỉ hiển thị các đối tượng trong tầm nhìn quanh camera và tạm ẩn các đối tượng ở ngoài, riêng phương tiện đang được người dùng điều khiển luôn được giữ hiển thị. Thứ ba là giảm số lệnh vẽ để hạ tải cho khâu kết xuất. Để chuyển động mượt khi tốc độ khung hình

cao, vị trí phương tiện được nội suy giữa hai bước mô phỏng liên tiếp ở phía hiển thị.

5.6.3 Kết quả đạt được

Nhờ các kỹ thuật trên, hệ thống duy trì được hiển thị ổn định khi số lượng phương tiện tăng, đồng thời chuyển động mượt mà ngay cả khi tốc độ khung hình cao hơn nhịp cập nhật của mô phỏng.

5.7 Kết chương

Chương này đã trình bày các giải pháp và đóng góp nổi bật của đồ án: hai thuật toán dựng đoạn đường và nút giao ba chiều bám sát dữ liệu, dùng chung cho mạng sinh tự động và bản đồ thực tế; cơ chế tạo phương tiện của người dùng và chiếm quyền điều khiển dựa trên máy trạng thái và bật tắt vật lý theo trạng thái; hệ thống va chạm giả lập tai nạn; cơ chế lưu trữ phiên và phát lại; giao tiếp thời gian thực tin cậy bằng tách khung bản tin; và các kỹ thuật tối ưu hiệu năng hiển thị. Các giải pháp này vừa giải quyết các vấn đề kỹ thuật cụ thể, vừa tạo nên những tính năng làm nên giá trị của hệ thống. Chương tiếp theo tổng kết kết quả đạt được và đề xuất hướng phát triển.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong phạm vi đề tài, đồ án đã xây dựng được một hệ thống mô phỏng và hiển thị giao thông trong không gian ba chiều, trong đó SUMO đảm nhiệm mô phỏng vi mô, Python điều khiển mô phỏng theo từng bước thời gian thông qua TraCI, và Unity đảm nhiệm dựng cảnh, hiển thị theo thời gian thực và xử lý tương tác. So với cách quan sát kết quả mô phỏng truyền thống vốn chủ yếu dựa trên góc nhìn 2D từ trên cao, hệ thống của đồ án không chỉ tự động hóa luồng dữ liệu và đồng bộ theo bước mô phỏng mà còn cho phép người dùng trải nghiệm theo góc nhìn của người tham gia giao thông, qua đó hỗ trợ quan sát và phân tích trực quan hơn trong các kịch bản có mật độ đối tượng cao hoặc khi cần thuyết minh kết quả.

Về kết quả đạt được, đồ án đã hoàn thành các nhóm chức năng chính sau:

- **Kết nối và đồng bộ thời gian thực:** triển khai kênh giao tiếp giữa Unity và Python qua TCP cùng cơ chế tách khung bản tin bằng dấu hiệu kết thúc <END> để khắc phục đặc thù truyền theo luồng của TCP; hỗ trợ đồng thời bản tin dữ liệu trạng thái và bản tin điều khiển.
- **Tự động dựng cảnh từ dữ liệu mạng đường:** dựng mạng đường trong không gian ba chiều từ kịch bản sinh tự động theo lưới mê cung phục vụ kiểm thử, hoặc từ bản đồ thực tế OpenStreetMap, giảm đáng kể thao tác thủ công khi thay đổi kịch bản; với bản đồ OpenStreetMap còn trích xuất và dựng khối công trình (toà nhà) từ dữ liệu nguồn nhằm tăng tính trực quan của khung cảnh.
- **Hiển thị đa đối tượng theo thời gian:** tạo, cập nhật và thu hồi phương tiện, đèn tín hiệu và người đi bộ theo vòng đời mô phỏng; áp dụng nội suy chuyển động để hiển thị mượt khi tốc độ khung hình cao.
- **Tương tác lái xe:** cho phép người dùng chiếm quyền điều khiển một phương tiện và lái thủ công bằng cơ chế vật lý; xử lý va chạm để chuyển xe bị tông thành trạng thái “xác xe” gây ùn ứ; và trả quyền điều khiển để server xóa xe khỏi mô phỏng ngay lập tức, Unity tái sử dụng đối tượng khi xe vắng khỏi luồng tải xuống.
- **Hai chế độ vận hành và lưu trữ phiên:** hỗ trợ chế độ thời gian thực và chế độ phát lại; gom toàn bộ kết quả của mỗi lần chạy vào một thư mục phiên thống nhất (nhật ký hành trình, dữ liệu mạng, chuỗi trạng thái theo bước và bản tóm tắt), phục vụ truy vết và phát lại.

- **Điều khiển vận hành:** cung cấp các thao tác điều khiển camera, tạm dừng và tiếp tục, điều chỉnh tốc độ mô phỏng lúc đang chạy, và lọc đối tượng hiển thị theo vùng quan sát nhằm duy trì hiệu năng.
- **Tối ưu luồng dữ liệu và độ trễ:** thu gọn số lần gọi TraCI trong mỗi bước bằng cơ chế đăng ký (subscription) thay cho việc truy vấn riêng lẻ từng đối tượng, nén luồng dữ liệu trạng thái trước khi truyền và tắt cơ chế gộp gói (Nagle) để giảm độ trễ; đồng thời sửa lỗi mất khung dữ liệu khi các gói TCP dồn vào nhau và bổ sung cơ chế đo, hiển thị độ trễ đầu–cuối phục vụ đánh giá chất lượng đồng bộ.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần tiếp tục hoàn thiện. Thứ nhất, mặc dù luồng truyền thông đã được tối ưu một bước (gom truy vấn TraCI bằng subscription, nén dữ liệu trạng thái, giảm độ trễ gói tin), giao thức vẫn dựa trên định dạng văn bản JSON, nên với mạng có rất nhiều đối tượng vẫn còn dư địa cải thiện về tải xử lý và băng thông. Thứ hai, mô hình vật lý cho chế độ lái còn ở mức cơ bản, chưa phản ánh đầy đủ động lực học của phương tiện; khi trả quyền, xe bị xóa ngay thay vì được đưa trở lại dòng giao thông, do cơ chế tái nhập làn chưa được hiện thực. Thứ ba, hệ thống hiện chạy trên một máy và hỗ trợ một người dùng cho mỗi phiên mô phỏng, chưa hỗ trợ nhiều người dùng cùng tham gia một phiên. Thứ tư, hệ thống va chạm giả lập tai nạn hiện mới chỉ xử lý va chạm giữa xe với xe, chưa hỗ trợ va chạm giữa người đi bộ với xe, nên chưa tái hiện được đầy đủ các tình huống tai nạn liên quan đến người tham gia giao thông.

Từ quá trình thực hiện, đồ án rút ra một số kinh nghiệm chính: (i) trong hệ thống thời gian thực, cần thiết kế cơ chế phân định ranh giới thông điệp rõ ràng khi dùng TCP để tránh các lỗi khó truy vết; (ii) việc chuẩn hóa dữ liệu và thống nhất hệ tọa độ, đơn vị ngay từ đầu giúp giảm đáng kể công sức tích hợp giữa mô phỏng và hiển thị; và (iii) khi xe được điều khiển theo nhiều trạng thái khác nhau, việc quản lý vòng đời và quyền điều khiển một cách tập trung là then chốt để hệ thống vận hành ổn định.

6.2 Hướng phát triển

Trong thời gian tới, đồ án có thể được phát triển theo các hướng chính sau:

- **Mở rộng số loại phương tiện:** bổ sung thêm nhiều chủng loại phương tiện (như xe máy, xe tải, xe buýt) bên cạnh các loại hiện có, kèm theo mô hình ba chiều và đặc tính di chuyển riêng cho từng loại, nhằm phản ánh sát hơn sự đa dạng của dòng giao thông thực tế.
- **Tối ưu truyền thông và hiệu năng:** tiếp nối các tối ưu đã có (subscription,

nén dữ liệu, đo độ trễ đầu–cuối), giảm thêm chi phí đóng gói và phân tích bằng định dạng nhị phân thay cho văn bản JSON, chỉ truyền phần thay đổi theo từng bước (mã hóa sai phân), và lọc dữ liệu theo vùng quan sát để giảm tải khi số lượng đối tượng rất lớn.

- **Hỗ trợ nhiều người dùng đồng thời:** mở rộng kiến trúc để nhiều máy hiển thị cùng tham gia một phiên mô phỏng, mỗi người điều khiển một phương tiện riêng; đây là tính năng chưa có trong phiên bản hiện tại và đòi hỏi cơ chế đồng bộ quyền điều khiển và trạng thái giữa các máy.
- **Nâng cấp mô hình lái, trả quyền và xử lý va chạm:** cải thiện động lực học phương tiện cho chế độ lái thủ công; hiện thực cơ chế tái nhập làn khi trả quyền — định vị xe về làn gần nhất, cấp tuyến mới và trả lại cho SUMO thay vì xóa xe ngay, giúp xe hòa lại dòng giao thông tự nhiên hơn; đồng thời mở rộng hệ thống va chạm để xử lý cả va chạm giữa người đi bộ với xe, không chỉ giới hạn ở va chạm giữa xe với xe.
- **Mở rộng kịch bản và đánh giá định lượng:** xây dựng bộ kịch bản chuẩn theo nhiều kích thước mạng và mật độ phương tiện; chuẩn hóa các thước đo như tốc độ khung hình, độ trễ và thời gian di chuyển, đồng thời tự động hóa quá trình chạy thực nghiệm để có cơ sở so sánh khách quan.
- **Cải thiện chất lượng phần mềm:** bổ sung kiểm thử cho các mô-đun sinh mạng, sinh tuyến và đóng gói giao thức; chuẩn hóa cấu hình chạy và tài liệu hóa giao thức nhằm thuận tiện cho việc bảo trì và mở rộng về sau.

TÀI LIỆU THAM KHẢO

- [1] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent development and applications of SUMO – Simulation of Urban MObility,” in *The Fourth International Conference on Advances in System Simulation (SIMUL 2012)*, IARIA, 2012, pp. 128–138.
- [2] Eclipse Foundation, *Eclipse SUMO – simulation of urban MObility*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://eclipse.dev/sumo/>
- [3] OpenStreetMap contributors, *OpenStreetMap*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.openstreetmap.org>
- [4] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, “TraCI: An interface for coupling road traffic and network simulators,” in *Proceedings of the 11th Communications and Networking Simulation Symposium (CNS 2008)*, ACM, 2008, pp. 155–163.
- [5] Unity Technologies, *Unity user manual*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [6] Unity Technologies, *Rigidbody – unity scripting API*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.unity3d.com/ScriptReference/Rigidbody.html>
- [7] Unity Technologies, *WheelCollider – unity scripting API*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.unity3d.com/ScriptReference/WheelCollider.html>
- [8] J. Postel, “Transmission control protocol,” Internet Engineering Task Force, RFC 793, 1981. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc793>

PHỤ LỤC

A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH

A.1 Yêu cầu phần cứng và phần mềm

Hệ thống được phát triển và kiểm thử trên Windows 11 (64-bit) với hai cấu hình máy tính khác nhau, liệt kê trong Bảng A.1.

Bảng A.1: Cấu hình hai máy tính dùng để phát triển và kiểm thử

Thành phần	Máy bàn	Máy xách tay (Dell Inspiron 5620)
CPU	Intel Core i5-14400F (10 nhân)	Intel Core i5-1240P (12 nhân)
RAM	32 GB DDR4	16 GB DDR4
GPU	NVIDIA RTX 4060	NVIDIA MX570

Phần mềm không đặt ra yêu cầu phần cứng tối thiểu cụ thể; hiệu năng phụ thuộc chủ yếu vào số lượng phương tiện trong mô phỏng. Cả hai cấu hình trên đều vận hành hệ thống ở chế độ thời gian thực với kịch bản OSM thông thường mà không gặp vấn đề về hiệu năng.

Phần mềm bắt buộc:

- **Python 3.14 trở lên** — ngôn ngữ triển khai server mô phỏng. Tải tại <https://www.python.org/downloads/release/python-3140/>. Trong quá trình cài đặt, cần tích chọn “Add python.exe to PATH”. Nếu máy đã có nhiều phiên bản Python, cần đảm bảo Python 3.14 được ưu tiên trong biến môi trường PATH.
- **Eclipse SUMO** và thư viện **TraCI** — được cài tự động qua script `install.bat` (xem Mục A.2).
- **Microsoft Visual C++ Redistributable 2022 x64** — thư viện runtime cần thiết để chạy `sumo-gui.exe`. Cũng được cài tự động qua `install.bat`.
- **DirectX 11** trở lên — yêu cầu của bản build Unity. Đã có sẵn trên Windows 10/11; không cần cài thêm trên phần lớn máy tính hiện đại.

A.2 Cài đặt môi trường

Toàn bộ quá trình cài đặt được tự động hóa qua file `install.bat` nằm trong thư mục gốc của gói phần mềm. Script này thực hiện tuần tự các bước:

1. Kiểm tra Python 3.14 trở lên đã được cài và nằm trong PATH.
2. Tạo môi trường ảo `.venv` ngay trong thư mục gói (giúp cô lập hoàn toàn khỏi các phiên bản Python khác trên máy).
3. Nâng cấp `pip` và cài gói `eclipse-sumo` (bao gồm SUMO, TraCI và sumolib)

vào `.venv`.

4. Đăng ký thư mục `sumo/tools` vào `sys.path` của `.venv` qua file `sumo_tools.pth`, để `import traci` hoạt động mà không cần cài SUMO toàn cục (`eclipse-sumo` chứa TraCI trong `sumo/tools/` nhưng không tự thêm thư mục đó vào `sys.path`).
5. Kiểm tra và cài Microsoft Visual C++ Redistributable nếu chưa có.
6. Xác nhận `import traci` thành công và in đường dẫn `SUMO_HOME` để xác nhận toàn bộ chuỗi cài đặt hoạt động đúng.

Để chạy script, nhấp đúp vào `install.bat` hoặc thực thi lệnh sau trong Command Prompt tại thư mục gốc của gói phần mềm:

```
install.bat
```

Khi cài đặt thành công, script in ra phiên bản TraCI và đường dẫn `SUMO_HOME` như ví dụ dưới đây:

```
[OK] Python 3.14.3
[OK] .venv da tao xong.
[OK] Da dang ky: C:\Users\...\site-packages\sumo\tools
[OK] Visual C++ Redistributable da co san.
[OK] traci 1.27.0
[OK] SUMO_HOME: C:\Users\...\site-packages\sumo
```

Để xác nhận thủ công, chạy lệnh sau trong terminal tại thư mục gốc của gói:

```
.venv\Scripts\python.exe -c "import traci; print(traci.__version__)"
```

Nếu lệnh in ra số phiên bản mà không báo lỗi, môi trường đã sẵn sàng.

A.3 Khởi chạy hệ thống

Hệ thống hỗ trợ hai chế độ vận hành: mô phỏng thời gian thực và phát lại kịch bản đã ghi sẵn.

A.3.1 Mô phỏng thời gian thực

Chế độ này khởi động SUMO song song với Unity, truyền dữ liệu vị trí phương tiện theo từng bước mô phỏng. Thứ tự khởi chạy như sau.

Bước 1 — Khởi động server mô phỏng. Nhấp đúp vào file `Simulation.bat` (hoặc chạy trực tiếp trong terminal):

```
Simulation.bat
```


Giao diện launcher hiện ra với ba tab kịch bản:

- **Mô phỏng Maze (.map)** — mô phỏng trên bản đồ mê cung được tạo sẵn từ file `.map`.
- **Mô phỏng OSM (.osm)** — mô phỏng trên bản đồ thực tế lấy từ OpenStreetMap; hệ thống tự động chuyển đổi file `.osm` sang định dạng SUMO.
- **Kịch bản netedit** — chạy kịch bản do người dùng tự xây dựng bằng netedit, chỉ cần trỏ đến thư mục chứa `.net.xml` và `.rou.xml`.

Sau khi chọn tab và cấu hình các tham số, nhấn nút **Khởi động Server**. Launcher sẽ tự mở SUMO và bản build Unity.

Bước 2 — Kết nối Unity và bắt đầu mô phỏng. Trong cửa sổ Unity, nhấn **Mô phỏng thời gian thực**, sau đó nhấn **Lấy dữ liệu đường** và chờ thông báo xác nhận lấy dữ liệu thành công. Cuối cùng nhấn **Bắt đầu mô phỏng** để bắt đầu quan sát.

A.3.2 Phát lại kịch bản có sẵn (tiền kết xuất)

Chế độ tiền kết xuất không yêu cầu SUMO đang chạy — Unity đọc trực tiếp từ hai file JSON đã ghi sẵn trong quá trình mô phỏng.

Bước 1 — Chuẩn bị kịch bản. Một kịch bản hợp lệ gồm hai file:

- `road_data.json` — dữ liệu hình học mạng lưới đường.
- `scenario.json` — chuỗi trạng thái phương tiện theo từng bước.

Các file này được tạo tự động khi chạy mô phỏng thời gian thực và lưu vào thư mục `Server/result/`. Có thể dùng lại các kịch bản đã ghi trước đó.

Bước 2 — Mở bản build Unity và cấu hình đường dẫn. Mở file `TestGR1.1.exe` trong thư mục `UnityBuild/`, chọn **Tiền kết xuất**, nhập đường dẫn tuyệt đối của `road_data.json` vào ô “Đường dẫn bản đồ” và đường dẫn tuyệt đối của `scenario.json` vào ô “Đường dẫn kịch bản”.

Bước 3 — Phát lại. Nhấn nút **Phát lại**. Unity sẽ tải dữ liệu và bắt đầu tái hiện kịch bản mà không cần kết nối mạng hay SUMO.

A.4 Xử lý sự cố thường gặp

Lỗi kết nối TCP (“Connection refused” hoặc “Connection timed out”). Server Python và Unity giao tiếp qua các cổng TCP 5050, 5053 và 5054. Nếu gặp lỗi kết nối, cần kiểm tra: (i) launcher đã được khởi động trước Unity; (ii) tường lửa Windows không chặn các cổng trên; (iii) không có tiến trình SUMO hoặc Python nào từ phiên trước còn đang giữ cổng (có thể kết thúc qua Task Manager).

sumo-gui báo lỗi thiếu DLL. Nguyên nhân thường là thiếu Microsoft Visual

C++ Redistributable. Chạy lại `install.bat` để cài đặt tự động, hoặc tải thủ công tại https://aka.ms/vs/17/release/vc_redist.x64.exe.

import traci báo `ModuleNotFoundError`. Chạy lại `install.bat` để tạo lại `.venv` và cài lại gói `eclipse-sumo`. Lưu ý: `Simulation.bat` đã tự động dùng Python trong `.venv` — không cần thay đổi biến `PATH` sau khi cài xong.

Lỗi phiên bản Python không đúng (“Hệ thống yêu cầu Python 3.14+”). Tải Python 3.14 tại <https://www.python.org/downloads/release/python-3140/>, cài đặt với tùy chọn “*Add python.exe to PATH*”, sau đó mở *Environment Variables* → *System Variables* → *Path* và chuyển mục thư mục Python 3.14 lên trên các phiên bản cũ hơn.

B. ĐẶC TẢ USE CASE

Phụ lục này đặc tả chi tiết các use case còn lại của hệ thống, ngoài sáu use case nghiệp vụ chính đã được đặc tả trong Chương 2 (UC2, UC4, UC7, UC11, UC14, UC15). Mỗi use case được mô tả theo cùng một khuôn dạng gồm: tên, tác nhân, mô tả, tiền điều kiện, luồng sự kiện chính, luồng sự kiện phụ và hậu điều kiện. Tác nhân chính của hệ thống là Người dùng; trình mô phỏng SUMO tham gia với vai trò hệ thống ngoài ở những use case có tương tác với mô phỏng. Các hành vi tự động (lọc đối tượng hiển thị, xử lý va chạm) được mô hình hóa là quan hệ «include»/«extend» thay vì use case độc lập.

B.1 Đặc tả use case Tạo bản đồ từ lưới mê cung

Tên	Tạo bản đồ từ lưới mê cung
Tác nhân	Người dùng
Mô tả	Hệ thống sinh mạng đường của SUMO từ một tệp lưới ký tự, trong đó ô tường và ô lối đi được mô tả bằng các ký hiệu quy ước.
Tiền điều kiện	Người dùng có tệp lưới mê cung hợp lệ và đã chọn số làn đường.
Luồng chính	(1) Người dùng chọn tệp lưới và số làn. (2) Hệ thống đọc kích thước và ma trận lưới. (3) Hệ thống tạo nút giao cho các ô lối đi và sinh cạnh nối các ô kề nhau. (4) Hệ thống ghi các tệp mô tả nút, cạnh và điểm qua đường. (5) Hệ thống biên dịch thành tệp mạng đường hoàn chỉnh.
Luồng phụ	(2a) Tệp lưới sai định dạng: hệ thống báo lỗi và dừng. (5a) Biên dịch mạng thất bại: hệ thống báo lỗi và dừng.
Hậu điều kiện	Tệp mạng đường được tạo, sẵn sàng cho bước sinh tuyến.

B.2 Đặc tả use case Nạp kịch bản tự dựng từ netedit

Tên	Nạp kịch bản tự dựng từ netedit
Tác nhân	Người dùng
Mô tả	Người dùng cung cấp kịch bản do mình tự dựng bằng netedit; hệ thống xác thực và nạp vào để chạy mô phỏng mà không cần qua bước sinh tuyến tự động.
Tiền điều kiện	Người dùng đã dựng kịch bản bằng netedit và xuất ra thư mục chứa ít nhất tệp <code>.net.xml</code> và <code>.rou.xml</code> .
Luồng chính	(1) Người dùng chọn tab “Kịch bản netedit” trong giao diện khởi chạy. (2) Người dùng chọn thư mục chứa kịch bản. (3) Hệ thống kiểm tra sự tồn tại của <code>.net.xml</code> và <code>.rou.xml</code> . (4) Hệ thống sao chép các tệp vào thư mục chuẩn và ghi tệp cấu hình mô phỏng. (5) Hệ thống khởi chạy mô phỏng với kịch bản đã nạp.
Luồng phụ	(3a) Thư mục thiếu <code>.net.xml</code> hoặc <code>.rou.xml</code> : hệ thống báo lỗi và yêu cầu chọn lại.
Hậu điều kiện	Kịch bản do người dùng cung cấp được nạp thành công, sẵn sàng để chạy mô phỏng.

B.3 Đặc tả use case Sinh tuyến đường cho người đi bộ

Tên	Sinh tuyến đường cho người đi bộ
Tác nhân	Người dùng
Mô tả	Hệ thống sinh các tuyến di chuyển cho người đi bộ và thiết lập tham số hành vi chờ đợi tại điểm qua đường.
Tiền điều kiện	Đã có mạng đường có làn dành cho người đi bộ.
Luồng chính	(1) Người dùng bật tùy chọn sinh người đi bộ và đặt mức độ thiếu kiên nhẫn. (2) Hệ thống chọn các cặp điểm đầu–cuối cho người đi bộ. (3) Hệ thống ghi các tuyến người đi bộ vào tệp tuyến đường cùng tham số đã đặt.
Luồng phụ	(1a) Người dùng tắt tùy chọn người đi bộ: hệ thống bỏ qua bước sinh tuyến người đi bộ.
Hậu điều kiện	Tệp tuyến đường có thêm các tuyến người đi bộ.

B.4 Đặc tả use case Cấu hình tham số mô phỏng

Tên	Cấu hình tham số mô phỏng
Tác nhân	Người dùng
Mô tả	Người dùng thiết lập các tham số cho lần chạy thông qua giao diện khởi chạy.
Tiền điều kiện	Giao diện khởi chạy đã sẵn sàng.
Luồng chính	(1) Người dùng chọn nguồn bản đồ, số làn và số lượng cặp điểm đầu–cuối. (2) Người dùng chọn kiểu phân chia cặp điểm và bật/tắt người đi bộ. (3) Người dùng chọn chế độ kết xuất (thời gian thực hoặc tiền kết xuất); riêng thời gian thực chọn thêm chế độ hiển thị (chỉ 3D hoặc cả 2D và 3D). (4) Hệ thống ghi nhận cấu hình cho lần chạy.
Luồng phụ	(1a) Giá trị nhập không hợp lệ: hệ thống dùng giá trị mặc định hoặc yêu cầu nhập lại.
Hậu điều kiện	Cấu hình được lưu, sẵn sàng để khởi chạy mô phỏng.

B.5 Đặc tả use case Chạy mô phỏng chế độ phát lại

Tên	Chạy mô phỏng chế độ phát lại
Tác nhân	Người dùng, SUMO
Mô tả	Hệ thống chạy trước toàn bộ mô phỏng, ghi lại trạng thái từng bước, sau đó ứng dụng hiển thị đọc dữ liệu để phát lại.
Tiền điều kiện	Đã có kịch bản mô phỏng hợp lệ.
Luồng chính	(1) Người dùng chọn chế độ tiền kết xuất và khởi chạy. (2) Hệ thống lưu dữ liệu hình học mạng đường vào <code>road_data.json</code> . (3) Hệ thống chạy mô phỏng đến khi kết thúc, ghi trạng thái từng bước vào <code>scenario.json</code> . (4) Người dùng mở ứng dụng hiển thị, chọn chế độ tiền kết xuất, nhập đường dẫn đến <code>road_data.json</code> và <code>scenario.json</code> , rồi nhấn Phát lại .
Luồng phụ	(3a) Quá trình chạy bị ngắt giữa chừng: hệ thống lưu phần dữ liệu đã ghi được trước khi dừng.
Hậu điều kiện	Tập kịch bản được tạo và có thể phát lại nhiều lần mà không cần chạy lại mô phỏng.

B.6 Đặc tả use case Chạy kèm giao diện sumo-gui

Tên	Chạy kèm giao diện sumo-gui
Tác nhân	Người dùng, SUMO
Mô tả	Ở chế độ thời gian thực, người dùng chọn một trong hai chế độ hiển thị: chỉ ba chiều, hoặc đồng thời cả giao diện hai chiều của SUMO và hiển thị ba chiều để đối chiếu.
Tiền điều kiện	Mô phỏng chạy ở chế độ thời gian thực.
Luồng chính	(1) Người dùng chọn chế độ hiển thị “Cả 2D và 3D”. (2) Hệ thống khởi chạy SUMO ở chế độ có giao diện. (3) Cửa sổ hai chiều hiển thị đồng thời với hiển thị ba chiều.
Luồng phụ	(1a) Chọn chế độ “Chỉ 3D”: SUMO chạy ẩn, người dùng vẫn quan sát qua hiển thị ba chiều của Unity.
Hậu điều kiện	Người dùng quan sát mô phỏng qua chế độ hiển thị đã chọn.

B.7 Đặc tả use case Hiển thị phương tiện theo thời gian

Tên	Hiển thị phương tiện theo thời gian
Tác nhân	Người dùng
Mô tả	Ứng dụng hiển thị tạo, cập nhật và thu hồi các đối tượng phương tiện ba chiều theo dữ liệu trạng thái nhận được.
Tiền điều kiện	Mô phỏng đang chạy và dữ liệu trạng thái đang được truyền tới.
Luồng chính	(1) Ứng dụng nhận danh sách phương tiện theo từng bước. (2) Với phương tiện mới, ứng dụng tạo đối tượng và gán mô hình. (3) Với phương tiện đã có, ứng dụng cập nhật vị trí và hướng, đồng thời nội suy chuyển động giữa các bước. (4) Với phương tiện không còn xuất hiện, ứng dụng thu hồi đối tượng để tái sử dụng.
Luồng phụ	(3a) Số lượng phương tiện lớn: ứng dụng áp dụng tái sử dụng đối tượng để duy trì hiệu năng.
Hậu điều kiện	Cảnh ba chiều phản ánh đúng tập phương tiện hiện có và chuyển động của chúng.

B.8 Đặc tả use case Hiển thị đèn tín hiệu

Tên	Hiển thị đèn tín hiệu
Tác nhân	Người dùng
Mô tả	Ứng dụng hiển thị trạng thái đèn tín hiệu tại các nút giao theo dữ liệu trạng thái nhận được.
Tiền điều kiện	Mô phỏng đang chạy và mạng đường có đèn tín hiệu.
Luồng chính	(1) Ứng dụng nhận trạng thái đèn của từng nút giao theo từng bước. (2) Ứng dụng xác định vị trí đặt đèn dựa trên hình học làn đường được điều khiển. (3) Ứng dụng hiển thị màu đèn tương ứng với trạng thái đỏ, vàng hoặc xanh.
Luồng phụ	(1a) Nút giao không có đèn: ứng dụng bỏ qua, không hiển thị đèn cho nút đó.
Hậu điều kiện	Trạng thái đèn tín hiệu được hiển thị đồng bộ với mô phỏng.

B.9 Đặc tả use case Hiển thị người đi bộ

Tên	Hiển thị người đi bộ
Tác nhân	Người dùng
Mô tả	Ứng dụng hiển thị các đối tượng người đi bộ ba chiều di chuyển theo dữ liệu trạng thái nhận được.
Tiền điều kiện	Mô phỏng đang chạy và kịch bản có người đi bộ.
Luồng chính	(1) Ứng dụng nhận danh sách người đi bộ theo từng bước. (2) Ứng dụng tạo, cập nhật và thu hồi đối tượng người đi bộ tương tự như với phương tiện. (3) Ứng dụng thể hiện chuyển động của người đi bộ.
Luồng phụ	(2a) Khi mô phỏng tạm dừng, đối tượng người đi bộ phải đứng yên, không thực hiện chuyển động tại chỗ.
Hậu điều kiện	Người đi bộ được hiển thị đồng bộ với mô phỏng.

B.10 Đặc tả use case Điều khiển camera quan sát

Tên	Điều khiển camera quan sát
Tác nhân	Người dùng
Mô tả	Người dùng thay đổi góc nhìn quan sát cảnh ba chiều, gồm chế độ tự do và chế độ bám theo một phương tiện.
Tiền điều kiện	Cảnh ba chiều đang được hiển thị.
Luồng chính	(1) Người dùng chọn chế độ camera. (2) Ở chế độ tự do, người dùng di chuyển và xoay góc nhìn theo ý muốn. (3) Ở chế độ bám theo, camera đi theo phương tiện được chọn.
Luồng phụ	(3a) Phương tiện đang bám theo rời khỏi mô phỏng: camera chuyển về chế độ tự do.
Hậu điều kiện	Góc nhìn quan sát thay đổi theo lựa chọn của người dùng.

B.11 Đặc tả use case Điều chỉnh tốc độ mô phỏng

Tên	Điều chỉnh tốc độ mô phỏng
Tác nhân	Người dùng
Mô tả	Người dùng thay đổi tốc độ chạy của mô phỏng trong khi đang chạy.
Tiền điều kiện	Mô phỏng đang chạy ở chế độ thời gian thực.
Luồng chính	(1) Người dùng đặt giá trị giãn cách thời gian giữa các bước. (2) Hệ thống cập nhật tham số một cách an toàn cho tiến trình mô phỏng. (3) Mô phỏng chạy nhanh hơn hoặc chậm hơn theo giá trị mới.
Luồng phụ	(1a) Giá trị không hợp lệ: hệ thống bỏ qua và giữ nguyên tốc độ hiện tại.
Hậu điều kiện	Tốc độ chạy mô phỏng thay đổi theo thiết lập của người dùng.

B.12 Đặc tả use case Lọc đối tượng hiển thị theo vùng

Tên	Lọc đối tượng hiển thị theo vùng
Tác nhân	Người dùng
Mô tả	Ứng dụng chỉ hiển thị các đối tượng nằm trong vùng quan sát và tạm ẩn các đối tượng ở ngoài để duy trì hiệu năng.
Tiền điều kiện	Cảnh ba chiều đang hiển thị nhiều đối tượng.
Luồng chính	(1) Hệ thống xác định vùng quan sát quanh camera. (2) Hệ thống hiển thị các đối tượng trong vùng và tạm ẩn các đối tượng ngoài vùng. (3) Khi camera di chuyển, hệ thống cập nhật lại tập đối tượng được hiển thị.
Luồng phụ	(2a) Đối tượng đang được người dùng điều khiển không bị tạm ẩn dù ra ngoài vùng.
Hậu điều kiện	Số đối tượng được vẽ giảm đi, hiệu năng được duy trì.

B.13 Đặc tả use case Xử lý va chạm sinh xác xe

Tên	Xử lý va chạm sinh xác xe
Tác nhân	Người dùng, SUMO
Mô tả	Khi phương tiện do người dùng điều khiển va chạm với một phương tiện đang do SUMO điều khiển, phương tiện bị tông chuyển sang trạng thái xác xe gây ùn ứ và tự biến mất sau một số bước.
Tiền điều kiện	Người dùng đang điều khiển một phương tiện; tồn tại phương tiện khác do SUMO điều khiển.
Luồng chính	(1) Hệ thống phát hiện va chạm giữa hai phương tiện. (2) Hệ thống chuyển phương tiện bị tông sang trạng thái xác xe và bật vật lý để nó bị đẩy trong Unity. (3) Phía mô phỏng đặt tốc độ xe về 0 để tạo ùn ứ cho các phương tiện phía sau; đồng thời vẫn đồng bộ vị trí từ Unity sang SUMO nếu xác xe bị đẩy dịch chuyển do vật lý. (4) Sau một số bước mô phỏng định trước, hệ thống loại bỏ xác xe.
Luồng phụ	(4a) Mô phỏng tạm dừng: bộ đếm số bước dừng lại, việc loại bỏ xác xe cũng tạm hoãn.
Hậu điều kiện	Phương tiện bị tông trở thành xác xe rồi biến mất; dòng giao thông phản ánh tình huống ùn ứ.